

A project report on

AutoTasker: Self-Service Task Scheduler

Submitted in partial fulfillment for the award of the degree of

**Bachelor of Technology in Computer Science and Business
Systems**

by

Kirthick Easwar B (22BBS0126)



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

May 2026

AutoTasker: Self-Service Task Scheduler

Submitted in partial fulfillment for the award of the degree of

**Bachelor of Technology in Computer Science and Business
Systems**

by

Kirthick Easwar B (22BBS0126)



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

May, 2026

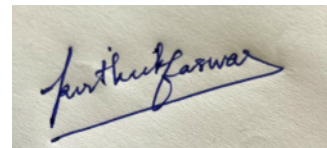
DECLARATION

I hereby declare that the thesis entitled “Auto Tasker: Self-Service Task Scheduler” submitted by Kirthick Easwar B, for the award of the degree of Bachelor of Technology in Computer Science and Business Systems, Vellore Institute of Technology is a record of bonafide work carried out by me under the supervision of Chidvilas Atkuri.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 21-05-2026

A handwritten signature in blue ink, reading "Kirthick Easwar B", written over a horizontal line.

Signature of the Candidate

Internship completion certificate

ABSTRACT

Modern enterprises rely on scheduled execution of recurring and non-recurring jobs for reporting, data movement, integration, and maintenance workflows. In many environments, scheduling is either script-centric or distributed across disconnected tools, which creates challenges in visibility, troubleshooting, role-based control, and user self-service. This project, AutoTasker, addresses these gaps by delivering a web-based self-service task scheduling platform that supports task lifecycle management, schedule control, execution tracking, and operational auditability.

The system introduces a clear role model with Viewer, Editor, and Admin privileges. Viewer users can monitor tasks and execution outcomes without mutation permissions. Editor users can create and update tasks and schedules and trigger-controlled actions. Admin users can manage platform users, task types, and environment migration operations such as export/import. The UI includes dedicated modules for dashboard metrics, task operations, audit trails, troubleshooting, and execution history.

A key contribution of this implementation is practical scheduling usability. The platform supports both one-time and recurring schedules through cron-driven definitions. To improve usability and reduce configuration errors, the system provides structured input support, cron validation, and natural-language schedule preview. The platform also supports role-based route and action gating, first-time password setup flow, task-type governance, and meaningful operational feedback through status chips, confirmations, and toast notifications.

The frontend is built using React, TypeScript, React Router, Material UI, and Vite. It is integrated with backend APIs through a structured client layer with typed request/response mappings and centralized error handling. Domain entities include Users, Task Types, Tasks, Schedules, Audit entries, and Task History records. The system captures both action-level audit events and run-level execution telemetry, enabling operational diagnosis and accountability.

Testing was performed across role flows, authentication branches, task/schedule operations, cron validation paths, and edge cases such as past-time non-recurring schedules, invalid cron expressions, and duplicate submissions. Results demonstrate that the solution provides a clean operational surface for scheduler control, improves visibility, and minimizes manual dependency on backend teams for routine task operations.

AutoTasker provides a scalable foundation for enterprise scheduling workflows with extensibility toward advanced capabilities such as queue management, retry policies, notifications, and analytics. The project demonstrates the practical value of combining robust scheduling semantics with role-governed UX and audit-first platform design.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Mr.Chidvilas Atkuri for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Software Engineering and Enterprise Automation.

I would like to express my gratitude to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Dr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G.V. Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr Jaisankar N Dean for providing an exceptional working environment and inspiring all of us during the tenure of the course

In jubilant mood I express ingeniously my whole-hearted thanks to Mythili T. Head of Department, Bachelor of Technology in Computer Science and Business Systems, all teaching staff and members working as limbs of our university for their not-self-centered enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project.

Place: Vellore

Date: 21-05-2026

Kirthick Easwar B

CONTENTS

LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ACRONYMS	ix
CHAPTER 1 INTRODUCTION	1
1.1 About the Organization	1
1.2 Introduction to AutoTasker	1
1.3 Problem Statement	3
1.4 Objectives of the Project	5
1.5 Scope of the Project	6
CHAPTER 2 PROJECT ACTIVITIES AND EVENTS	9
2.1 Induction and Onboarding	9
2.2 Project Planning and Requirement Analysis	10
2.3 Tools and Environment Setup	13
2.4 Development Workflow and Progress Reviews	16
2.5 Mid-Term and Final Review Presentations	19
CHAPTER 3 REQUIREMENT ANALYSIS	23
3.1 User Roles and Access Model	23
3.2 Functional Requirements	23
3.3 Non-Functional Requirements	26
3.4 Hardware Requirements	28
3.5 Software Requirements	29
CHAPTER 4 SYSTEM DESIGN	30
4.1 System Overview	30
4.2 System Architecture	31
4.3 Module Description	32
4.4 Data Model and Status Lifecycle	35
4.5 Use Case and Workflow Analysis	36
CHAPTER 5 IMPLEMENTATION	37
5.1 Authentication and First-Time Password Setup	37
5.2 Role-Based Dashboards and Navigation	38
5.3 Task and Schedule Management	39
5.4 Cron Builder and Natural Language Preview	41
5.5 Audit, Troubleshoot, and Task History	41
5.6 Admin Modules (Manage Users and Task Types)	42
5.7 Export/Import for Environment Migration	43
CHAPTER 6 RESULTS AND REVIEWS	45
6.1 Role-Based Flow Validation	45
6.2 Task Lifecycle and Schedule Validation	46
6.3 Audit and Task History Output Screens	47
6.4 UI/UX Improvements from Review Feedback	49
6.5 Observations and Outcomes	50
CHAPTER 7 USED TECHNOLOGIES	53
7.1 React	53
7.2 TypeScript	54
7.3 Material UI	55

7.4 React Router	56
7.5 Vite.....	57
7.6 Backend API Integration.....	57
7.7 Deployment and Version Control.....	59
CHAPTER 8 CONCLUSION AND FUTURE WORK.....	60
8.1 Conclusion	60
8.2 Limitations	60
8.3 Future Work.....	61
REFERENCES.....	62

LIST OF FIGURES

Figure No.	Title	Page No.
1	Existing Scheduling Workflow Overview	3
2	Current Batch Scheduling Dependency with Windows Scheduler	4
3	Proposed Self-Service Scheduling Workflow	5
4	User Role Access Matrix	24
5	Tech Stack and System Architecture Diagram	32
6	Backend ER Diagram	35
7	Use Case Diagram	36
8	Test Coverage Report	52

LIST OF TABLES

Table No.	Title
1	Project Planning
2	Tools and Environment Setup
3	Functional Requirements
4	Non-Functional Requirements
5	Hardware Requirements
6	Software Requirements
7	Module vs Project Component Mapping
8	Technical Stack Summary
9	User Roles and Access Permissions
10	Task Lifecycle Management Features
11	Scheduler Behaviour and Edge Case Handling
12	Database Entity Summary
13	API Controller Summary
14	Test Coverage Summary
15	Unit and Integration Testing Summary

LIST OF ACRONYMS

Acronym	Full Form
API	Application Programming Interface
UI	User Interface
RBAC	Role-Based Access Control
SQL	Structured Query Language
DB	Database
ER	Entity Relationship
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
KPI	Key Performance Indicator
SLA	Service Level Agreement
IDE	Integrated Development Environment
OS	Operating System

RAM	Random Access Memory
SSD	Solid State Drive
QA	Quality Assurance
UAT	User Acceptance Testing
XML	Extensible Markup Language
CLI	Command Line Interface
GUI	Graphical User Interface
DLL	Dynamic Link Library
PDF	Portable Document Format

Chapter 1

1.1 ABOUT THE ORGANIZATION

ION Trading is a global financial technology company that provides mission-critical trading, analytics, and risk management solutions to financial institutions, corporations, and governments worldwide. Founded in 1999 and headquartered in Dublin, Ireland, ION Trading has grown into one of the most trusted names in capital markets technology, serving over 40,000 clients across more than 50 countries.

ION Trading operates across several key business verticals including fixed income trading, derivatives, commodities, treasury management, Foreign Exchange and enterprise resource planning. The company's product suite spans electronic trading platforms, trade execution and order management systems, post-trade processing, and real-time risk analytics. These solutions are designed to streamline complex financial workflows, reduce operational risk, and deliver measurable efficiency gains for some of the world's most sophisticated financial institutions.

The organization is structured around multiple specialized divisions — ION Markets, ION Treasury, ION Corporates, and ION Analytics — each catering to a distinct segment of the financial services industry. ION Markets focuses on capital markets trading solutions, ION Treasury provides treasury and risk management platforms, ION Corporates delivers enterprise financial software, and ION Analytics offers data and risk intelligence tools.

My internship project was carried out at ION Group from 19 January 2026 to 19 May 2026 under the guidance of Mr. Chid Vilas Atkuri (Mentor) and Mr. Vaibhav Nagpal (Manager). During this period, I worked on the design and implementation of Self-Service Task Scheduler, a role-based web utility that allows users to create, monitor, troubleshoot, and manage scheduled tasks through a centralized interface.

The project focused on building a clean, production-oriented frontend integrated with backend APIs, with key emphasis on role-based access (Viewer/Editor/Admin), task and schedule lifecycle management, cron-based recurring scheduling, audit and execution history visibility, and operational features such as retry, pause/resume, and export/import support. This work aligned with ION's broader engineering goal of improving operational efficiency through controlled self-service automation.

1.2 INTRODUCTION TO THE PROJECT

AutoTasker: Self-Service Task Scheduler is a complete role-based scheduling platform developed to streamline the management of operational jobs in enterprise environments. In many organizations, recurring and one-time tasks are often controlled through scripts, fragmented tools, and manual backend coordination. This leads to delays in execution

changes, limited operational visibility, and increased dependency on technical teams for routine actions. AutoTasker addresses this gap by providing a unified system through which users can create, configure, monitor, and troubleshoot scheduled tasks in a controlled and traceable manner.

The project has been implemented as an end-to-end solution comprising a web application layer, backend API services, and persistent data storage. The application layer supports task and schedule operations through a structured user interface, while backend services manage authentication, task lifecycle operations, schedule execution interactions, audit tracking, and run history retrieval. The data layer maintains system entities such as users, task types, tasks, schedules, audits, and execution history records. This integrated approach ensures that operational workflows are both user-friendly and technically reliable.

A key design feature of AutoTasker is role-based access control with three user levels: Viewer, Editor, and Admin. Viewer users are restricted to monitoring and review activities, including access to tasks, audit records, and execution history. Editor users are authorized to perform operational actions such as creating and updating tasks and schedules. Admin users hold full system-level privileges, including user management, task type management, and configuration migration support. This model ensures separation of responsibilities and reduces the risk of unauthorized modifications.

The platform supports complete task lifecycle management, including task creation, update, manual execution, pause/resume control, and deletion. It also supports both recurring and non-recurring schedules. Recurring schedules are configured using cron expressions, while non-recurring schedules are configured using date and time inputs.

To reduce configuration errors, the system validates cron expressions before submission and provides natural-language interpretation of schedule logic. Non-recurring schedules are validated to prevent invalid past-time entries. These controls improve usability while preserving scheduler correctness.

To strengthen observability and operational diagnosis, AutoTasker includes dedicated modules for **Audit** and Task History. The Audit module captures user and system actions such as creation, updates, status changes, and manual run triggers. The Task History module captures execution-level outcomes, including run status, start and finish time, exit code, and output snippet. In addition, the Troubleshoot module focuses on error-state tasks and provides retry capability for authorized users. Administrative utility is further extended through export and import functionality, enabling task configuration transfer across environments.

In summary, AutoTasker delivers a complete and production-oriented scheduling system that improves operational efficiency, reduces manual dependency, and enforces controlled access with full traceability. The project demonstrates a practical balance between usability, governance, and technical integration, making it suitable for real-world operational scheduling requirements.

1.3 PROBLEM STATEMENT

In enterprise operations, scheduled tasks are used for recurring business processes, one-time utility jobs, and periodic system activities. Even though these tasks are operationally critical, their management is often fragmented across multiple channels such as scripts, backend requests, manual escalation, and ad-hoc communication. This fragmented setup creates inefficiency, delays, and limited control for functional users who need quick operational actions.

The first core problem is the **absence of a unified self-service interface** for scheduler operations. Users who are responsible for monitoring and maintaining task health often cannot independently perform simple but important actions such as creating a task, modifying a schedule, pausing an active flow, or retrying a failed execution. These actions usually require support from technical teams, which increases turnaround time and creates avoidable operational dependency.



Figure 1: Existing Scheduling Workflow Overview

The second major issue is weak observability for action and execution tracking. When failures occur, users need clear answers to practical questions: what changed, who changed it, when it changed, and what happened during execution. In traditional setups, this information is not presented in a consolidated and user-friendly manner. As a result, troubleshooting becomes slow, root-cause understanding is delayed, and decision-making quality is reduced during incident response.



Figure 2: Current Batch Scheduling Dependency with Windows Scheduler

The third problem is **complexity in recurring schedule configuration**. Cron expressions are highly flexible but are difficult for many users to configure correctly without support. Misconfigured expressions can lead to missed runs, excessive runs, or unintended execution windows. Without strong validation and understandable schedule preview, such errors can propagate into production workflows and affect reliability.

The fourth challenge is **governance and access control**. Different operational roles require different permissions: some users only need visibility, while others need controlled modification rights. If role boundaries are not enforced at both route and action level, it can result in unauthorized changes, accidental disruptions, and audit concerns. Therefore, scheduler operations must be controlled through a clear role model with strict permission mapping.

The fifth problem concerns **consistency and usability of operational workflows**. In the absence of a standardized platform, similar actions are performed differently by different users, leading to inconsistent outcomes and higher training overhead. A well-structured scheduler system should provide predictable workflows with clear validation, confirmation controls, and meaningful feedback for both success and failure states.

Hence, the project addresses the need for a complete, role-based, and operationally safe task scheduling platform that centralizes task and schedule management, improves observability through audit and execution history, reduces manual dependency on backend teams, and increases reliability through validation-first scheduling and controlled self-service operations.

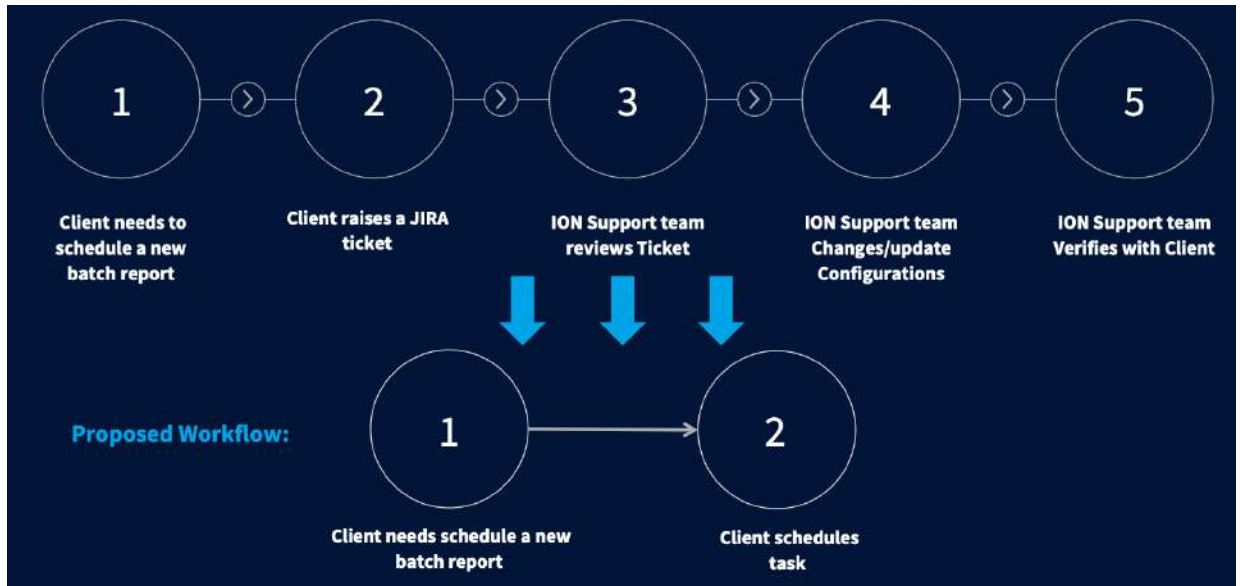


Figure 3: Proposed Self-Service Scheduling Workflow

1.4 OBJECTIVES OF THE PROJECT

The primary objective of this project is to design and implement AutoTasker, a complete self-service task scheduling platform that improves operational efficiency, governance, and visibility in day-to-day task execution workflows. The objectives are defined as follows:

O1. Centralize scheduler operations in a single platform.

Develop a unified system where users can perform all core scheduling activities such as task creation, schedule configuration, execution control, and monitoring without relying on fragmented tools or manual backend coordination.

O2. Implement strong role-based access control.

Establish a clear permission model with Viewer, Editor, and Admin levels, ensuring that each role can access only the routes and actions relevant to its operational responsibility.

O3. Provide complete task lifecycle management.

Enable creation, modification, execution, pause/resume, status updates, and logical deletion of tasks through controlled workflows with confirmation and validation support.

O4. Support both recurring and non-recurring scheduling requirements.

Allow flexible schedule definition using cron-based recurring expressions and date-time-based non-recurring scheduling, covering routine and one-time operational use cases.

O5. Reduce scheduling errors through validation-first design.

Validate cron expressions and schedule constraints before submission, and provide human-readable schedule interpretation to help users verify intent prior to saving.

O6. Improve troubleshooting and operational response time.

Provide dedicated troubleshooting support for error-state tasks and authorized retry capabilities to reduce downtime and speed up operational recovery.

O7. Ensure action traceability and execution transparency.

Implement separate but linked modules for action-level audit tracking and run-level task history so users can understand both what was changed and what execution outcome occurred.

O8. Build a production-ready and user-friendly interface.

Deliver responsive screens, consistent status indicators, clear error messages, loading and empty states, and duplicate-submission prevention for reliable day-to-day use.

O9. Integrate cleanly with backend services and data models.

Use typed API integration and modular architecture to align frontend behavior with backend contracts for users, task types, tasks, schedules, audits, and history records.

O10. Provide administrative governance and migration support.

Enable admin-level management of users and task types, along with export/import capability for controlled movement of task configurations across environments.

In summary, the project objectives are focused on delivering an operationally safe, role-aware, and scalable scheduler platform that reduces manual dependency, increases system clarity, and supports reliable enterprise task operations.

1.5 SCOPE OF THE PROJECT

The scope of this project is defined to cover complete scheduler operations from an end-user and administrator perspective through an integrated web platform and backend API ecosystem. It includes functional delivery, role-based governance, operational observability, and controlled configuration movement.

1.5.1 In-Scope Components

S1. Authentication and access control

- User login flow with role resolution
- First-time password setup flow for newly created users
- Password change capability for existing users
- Role-based route and action restrictions for Viewer, Editor, and Admin roles

S2. Task type governance (Admin)

- Creation, update, listing, and deletion of task types
- Management of task-type execution path metadata
- Backend-constrained update behavior where applicable

S3. Task lifecycle operations

- Create, view, update, run, pause/resume, and logical delete task flows
- Task metadata management including name, description, and email list
- Correct status transitions aligned with backend rules such as Active, Not Scheduled, Error, and Deleted visibility filtering

S4. Schedule lifecycle operations

- Add, update, pause/resume, and delete schedules under tasks
- Recurring schedule configuration using cron expression workflow
- Non-recurring schedule configuration using date and time input
- Validation rules to prevent invalid schedule creation such as invalid cron and past-time non-recurring entries

S5. Observability and operational diagnosis

- Audit module for tracking user/system actions
- Task History module for execution-level output including status, timing, exit code, and output snippet
- Troubleshoot module focused on error-state tasks with role-aware retry support

S6. Administrative utility and migration

- User management module including create, list, and delete
- Export module for task configuration extraction
- Import module for controlled task configuration ingestion in another environment

S7. Usability and reliability controls

- Confirmation dialogs for sensitive actions
- Loading, empty, and error states across views
- Toast-based feedback for success and failure outcomes
- Disable-on-submit behavior for API-triggering buttons to prevent duplicate requests

1.5.2 Out-of-Scope Components

The following are intentionally outside the scope of this project report and implementation phase:

- Internal infrastructure deployment details and environment secrets
- Proprietary operational and ticketing workflows specific to internal teams
- Deep backend scheduler engine internals beyond exposed API behavior
- Advanced external notification integrations as mandatory features
- Organization-specific confidential business logic not required for academic demonstration

1.5.3 Scope Boundaries and Limitations

The project focuses on controlled scheduler platform behavior exposed through approved backend APIs

Security and permission enforcement are implemented at application level and are expected to be complemented by backend enforcement

The report documents architecture and workflow behavior without exposing confidential operational content

The delivered system is designed to be extensible for future features such as advanced retry strategies, analytics, notification pipelines, and richer policy controls

Overall, the scope ensures that AutoTasker is presented as a complete, practical, and production-oriented scheduler solution while maintaining clear academic boundaries and confidentiality compliance.

CHAPTER 2

2.1 Induction and Onboarding

The internship began with a well-structured induction process, which played an important role in helping me understand the organization, its work environment, and its expectations from interns. The induction sessions introduced me to the company's work culture, professional standards, communication practices, reporting structure, and delivery expectations. These sessions helped me understand how work is planned, assigned, reviewed, and delivered in an enterprise environment.

During this phase, I realized that the internship project was not just a classroom-based implementation or a basic academic assignment. Instead, it was a practical, product-oriented solution designed to address real operational challenges. The project required me to think beyond simple feature development and focus on usability, reliability, maintainability, and long-term value.

After completing the initial induction, I was introduced to the team members who would guide and support me throughout the project. Regular check-ins were scheduled with my mentor and manager to review the progress of the project in short and consistent cycles. These discussions helped me understand whether the project was moving in the right direction and also gave me opportunities to receive feedback, clarify doubts, and improve the implementation approach.

This stage also helped me understand the major difference between academic implementation and enterprise-level implementation. In academic projects, the main focus is usually on making the feature work correctly. However, in an enterprise environment, the solution must be dependable, secure, scalable, easy to understand, and suitable for users with different roles and responsibilities. The interface should be simple enough for non-technical users, while the system should still maintain proper control, traceability, and governance.

As part of the onboarding process, I also studied the scheduler problem domain in detail. I understood how recurring and non-recurring tasks are commonly used in operational systems and how business users depend on these scheduled activities for routine processes. I also learned about the common issues that occur in scheduling systems, such as task failures, lack of visibility, delayed troubleshooting, dependency on technical teams, and difficulty in tracking execution history.

This understanding helped me identify the need for a self-service scheduling utility that could reduce dependency on IT teams and improve operational efficiency. The domain study gave a clear direction to the AutoTasker solution by highlighting the importance of visibility, access control, execution monitoring, audit history, and user-friendly design.

A key learning from this phase was that user experience and governance are equally important as technical correctness. A system may work technically, but it will not be

effective unless users can understand it easily, operate it confidently, and track actions clearly. Therefore, the project had to balance technical implementation with practical usability and enterprise control.

By the end of the onboarding phase, the initial project direction was clearly defined. The main objectives were to build a role-based self-service scheduler utility, ensure proper separation between monitoring users and editing users, integrate the frontend with backend APIs instead of depending only on mock behavior, keep the interface simple for non-technical operational users, and provide traceability through audit logs and execution history.

Overall, the onboarding phase created a strong foundation for the later stages of the project. It helped me understand the business problem, user expectations, technical requirements, and enterprise standards. This clarity supported better planning, design, and execution during the development phase of the AutoTasker project.

2.2 Project Planning and Requirement Analysis

After the onboarding phase was completed, the next stage of the internship focused on project planning. This planning was carried out using a requirement-first approach, where the main priority was to understand the actual needs of the users before moving into development. Instead of directly starting with screens or technical implementation, the planning process focused on identifying what users need to perform on a daily basis, what problems they currently face, and what controls must be included to prevent incorrect usage or unauthorized actions.

The requirements were gathered through guided discussions, workflow analysis, and continuous clarifications during the implementation process. Each requirement was studied from both a user perspective and a system perspective. From the user perspective, the focus was on making the application simple, clear, and useful for daily operational work. From the system perspective, the focus was on access control, data consistency, backend integration, error handling, and maintainability.

The first major planning decision was to clearly define the user personas and their permissions. Since the AutoTasker system was designed for operational users with different responsibilities, it was important to ensure that every user could access only the features relevant to their role. Three levels of access were finalized for the system.

The first role was the Viewer. A Viewer was given read-only access to the system. This role was mainly intended for users who only need to monitor tasks, schedules, execution status, and history without making any changes. Viewers can observe the system and understand task progress, but they cannot create, update, pause, resume, or delete any task or schedule.

The second role was the Editor. An Editor was given access to operational task and schedule actions. This role was intended for users who are responsible for managing

scheduled activities. Editors can create tasks, update task details, manage schedules, pause or resume schedules, delete schedules, and perform other day-to-day operations. However, their access is limited to operational activities and does not include full administrative control.

The third role was the Admin. An Admin was given full governance access across the system. This role included the ability to manage users, task types, permissions, administrative modules, and overall system configuration. Admin access was planned carefully because this role directly affects system control and governance. Therefore, admin features were separated from normal operational features to maintain clear responsibility boundaries.

This user role decision had a direct impact on both frontend and backend planning. On the frontend, it influenced routing, menu visibility, button access, page-level restrictions, and action-level restrictions. On the backend integration side, it influenced how API responses, permissions, and user session details would be handled. Every screen and every major action in the application was mapped against the three roles so that access control would remain consistent throughout the system.

The second major planning area was feature grouping. Instead of building the application as a collection of unrelated pages, the system was divided into clear operational modules. This helped make the application more structured, maintainable, and easy to understand. The first module was authentication and password setup, which handled user login, password creation, password reset flow, and secure access into the application.

The second module was task and schedule management. This module formed the core part of the AutoTasker solution. It included the creation and management of tasks, schedule creation, recurring and non-recurring schedule handling, pause and resume actions, deletion behavior, and status tracking. Since tasks and schedules are closely connected, their workflows were planned together to avoid confusion in the user interface.

The third module was the audit and troubleshoot flow. This module was planned to help users understand what happened in the system and why a task or schedule behaved in a particular way. Audit logs help track user actions, while troubleshoot information helps identify problems related to task execution, schedule failures, or backend response errors. This module was important because operational systems require visibility and traceability.

The fourth module was task execution history. This module was planned to provide users with a clear view of previous task runs, execution status, success or failure details, timestamps, and related messages. Execution history allows users to verify whether scheduled tasks are running as expected and also supports troubleshooting when an issue occurs.

The fifth module was admin management. This included user management, task type management, and other governance-related controls. Since these features affect system-wide configuration, they were planned separately and restricted to Admin users only.

The sixth module was the export and import utility. This feature was planned to support environment migration and easier data handling. Export and import functionality helps move or manage configuration data more efficiently and reduces manual effort during setup or migration activities.

The third major planning area was status modeling. Since AutoTasker deals with tasks, schedules, executions, and user actions, it was important to define status behavior correctly. Task and schedule states were carefully aligned with backend behavior so that the frontend would display correct information and allow only valid actions. Without proper status modeling, users could be shown incorrect buttons or misleading information.

Special attention was given to edge cases such as deleted entities, completed non-recurring schedules, paused schedules, and tasks that were created but not yet scheduled. For example, if a schedule is paused, the UI must clearly show that it is inactive and should provide the correct option to resume it. If a non-recurring schedule is completed, the system should not treat it like an active recurring schedule. Similarly, if a task exists without any schedule, it should move into a proper not-scheduled state instead of appearing as an active scheduled task.

The fourth major planning area was error handling and validation behavior. Since the application is intended for operational users, error messages had to be clear, direct, and useful. The requirement was not just to display backend error messages as they are, but to convert them into meaningful user-facing messages. Users should be able to understand what went wrong and what action they need to take next.

This planning influenced multiple parts of the application, including toast messages, form validations, confirmation dialogs, retry behavior, and empty-state messages. For example, if a user tries to create a schedule without entering required details, the system should clearly mention which field is missing. If an API request fails, the system should notify the user in a simple way instead of showing a technical error. If no execution history is available, the system should show a proper empty message rather than leaving the screen blank.

To ensure smooth implementation, a phased delivery approach was defined. This helped avoid unstable integration and allowed each part of the application to be developed, tested, reviewed, and improved step by step.

Phase 1 focused on the base application structure, authentication, and role gating. This phase included setting up the project structure, routing, login flow, session handling, and role-based access control. It created the foundation required for all later modules.

Phase 2 focused on task and schedule lifecycle implementation. This phase included creating tasks, updating tasks, deleting tasks, creating schedules, pausing and resuming schedules, and handling different task and schedule statuses. Since this was the core functionality of the system, it required careful mapping between frontend behavior and backend APIs.

Phase 3 focused on audit, troubleshoot, and history modules. This phase added visibility into system activity and task execution. It helped users track what actions were performed, view past executions, and understand failure details.

Phase 4 focused on admin features and environment migration tools. This included user management, task type management, and export/import-related features. These features supported governance and configuration management within the system.

Phase 5 focused on quality hardening, user experience polish, and bug fixes. In this phase, the application was reviewed for usability, consistency, validation issues, error handling, page responsiveness, and integration stability. Minor defects were fixed, and the overall user experience was improved.

This requirement-first and phased planning approach helped maintain steady progress throughout the project. It also allowed continuous requirement refinement without disturbing already completed modules. By planning user roles, feature modules, status behavior, validation rules, and delivery phases clearly, the development process became more organized and reliable. Overall, this stage played a key role in converting the initial project idea into a structured implementation plan for the AutoTasker system.

2.3 Tools and Environment Setup

After the planning phase was completed, the next stage of the project focused on setting up the development environment. This phase was important because a properly configured environment helps improve development speed, code quality, integration stability, and long-term maintainability. Since AutoTasker was planned as a practical enterprise-oriented application, the environment had to support fast iteration as well as clean integration with backend APIs.

The primary frontend technology stack used for the project included React, TypeScript, React Router, Material UI, and Vite. React was selected for building a component-based user interface, where different sections of the application could be developed as reusable and independent components. TypeScript was used to add strong typing to the frontend code, which helped reduce runtime errors and made the code easier to understand and maintain. React Router was used to manage navigation between different pages and modules. Material UI was used to create a clean, consistent, and responsive interface with ready-to-use components. Vite was used as the build tool because it provides fast project startup, quick refresh during development, and a smooth development experience.

The environment setup began with local project initialization and dependency installation. The required libraries and packages were installed based on the planned architecture of the application. After this, the project folder structure was aligned according to the functional modules of the system. A feature-based structure was followed so that each major module could be developed, tested, and maintained independently. This approach made the codebase more organized and reduced confusion during development.

The folder structure separated page-level logic, shared components, API services, utility functions, configuration files, and type definitions. Shared components such as buttons, forms, tables, dialogs, loaders, and status indicators were separated from individual page logic. This allowed commonly used UI elements to be reused across multiple screens. Utility functions were also separated to avoid repeating the same logic in different parts of the application. This structure improved readability and supported long-term scalability.

Route architecture was also set up during this phase. Since the application had multiple modules such as authentication, task management, schedule management, audit logs, execution history, troubleshooting, and admin features, proper routing was necessary. Each route was mapped to its respective page or module. Role-based routing was also considered so that users could only access the pages allowed for their role. This ensured that navigation was aligned with the permission model planned earlier.

A major activity during this phase was API integration setup. Initially, parts of the application were designed using mock behavior to understand the expected flow and screen structure. However, the project was later moved toward actual backend endpoint integration. This was important because the final application had to work with real data, real status changes, and real backend validation rules. Moving from mock-driven behavior to backend-driven behavior helped make the system more realistic and production-like.

To support clean API integration, a typed API client layer was prepared. This layer was responsible for handling communication between the frontend and backend services. Request and response models were mapped in a controlled manner so that API calls would not be scattered throughout the application. This made the code easier to debug and maintain. It also helped in handling backend responses such as success messages, validation errors, authorization failures, and status updates in a consistent way.

The use of TypeScript made API integration more reliable because request and response data structures could be clearly defined. For example, task details, schedule details, user roles, execution history records, audit entries, and error responses were handled using typed models. This reduced the chances of incorrect data usage in the frontend and helped ensure that the UI displayed backend information correctly.

Configuration management was another important part of the setup phase. Environment variables were used to manage values such as the API base URL. This allowed the

application to switch between local development, testing, and deployment environments without changing the source code. By keeping environment-specific details outside the main codebase, the application became easier to configure and safer to maintain.

Backend communication behavior was also validated during this phase. Since the frontend had to interact with backend APIs, checks were done to ensure that cross-origin behavior, request formats, response formats, and backend contract compatibility were working properly. This was necessary to confirm that all screens would remain functional when connected to real API calls instead of mock data.

Special attention was given to the schedule-related setup because scheduling is the core function of the AutoTasker system. Cron handling utilities were prepared to support recurring schedule configuration. These utilities helped validate cron expressions before they were submitted to the backend. This prevented users from submitting invalid schedule patterns and reduced the possibility of backend errors.

Apart from validation, user-friendly interpretation of cron expressions was also considered. Since cron expressions can be difficult for non-technical users to understand, the system needed to display schedule information in a clearer format wherever possible. This helped improve usability and reduced confusion for operational users.

Non-recurring schedule handling was also planned carefully. For one-time schedules, the system had to protect against invalid date and time inputs. This included preventing past date-time values, missing fields, or incorrect formats. Timezone-safe behavior was also considered so that the frontend and backend would interpret schedule times consistently. This was important because incorrect timezone handling can cause scheduled tasks to run at the wrong time.

Reliability safeguards were added during the setup phase itself. Buttons and actions connected to backend API calls were designed to disable during request execution. This prevented users from clicking the same action multiple times while the request was still processing. Such safeguards helped avoid duplicate submissions, repeated task creation, accidental repeated pause or resume actions, and unnecessary backend calls.

Error handling patterns were also prepared as part of the setup. API failures, validation errors, unauthorized access, and loading states were planned in a consistent way. This ensured that users would receive proper feedback when something went wrong. Instead of leaving screens blank or showing technical messages, the application was designed to provide clear and meaningful responses through alerts, toast messages, and form-level validations.

Overall, the development environment setup phase created a strong technical foundation for the AutoTasker project. It ensured that the project was not built using temporary prototype shortcuts, but followed a cleaner and more production-oriented structure. By setting up routing, folder structure, typed API integration, environment configuration,

cron utilities, validation patterns, and submission safeguards early, the later development phases became smoother and more reliable.

This setup phase reduced future rework and allowed the application to grow in a structured way. It also helped maintain consistency across modules and supported faster development during task management, schedule management, audit, history, troubleshooting, and admin feature implementation.

2.4 Development Workflow and Progress Reviews

The project was executed using an iterative development approach with continuous review and feedback. Instead of developing the complete application and presenting it only at the final stage, the work was divided into smaller functional cycles. Each feature was implemented, reviewed, refined, and then revalidated before moving to the next part of the system. This method helped maintain steady progress and ensured that issues were identified early rather than at the end of the project.

The iterative approach was especially useful because AutoTasker was not a simple static application. It involved user roles, backend API integration, task lifecycle flows, schedule state transitions, audit tracking, execution history, and admin-level controls. Since these features were connected to each other, a change in one area could affect another area. Regular review cycles helped reduce this risk by allowing each flow to be checked carefully after implementation.

The implementation started with authentication and role-controlled routing. This was an important first step because all later modules depended on proper access control. The login flow was connected with role-based behavior so that users could access only the pages and actions allowed for their role. Viewer, Editor, and Admin access levels were handled through routing rules, page restrictions, and action visibility. Once this role behavior became stable, the remaining features were built on top of this foundation.

After authentication and role gating were completed, the core scheduler functions were implemented. The first focus was on task management because tasks form the base entity of the system. Features such as task creation, task editing, task deletion, and task run controls were developed and connected with backend APIs. During this stage, care was taken to ensure that the frontend behavior matched backend expectations, especially for task status, validation, and user permissions.

Once the task lifecycle became stable, schedule-related operations were added. These included adding schedules, editing schedules, pausing schedules, resuming schedules, and deleting schedules. Since schedules could be recurring or non-recurring, additional logic was required to handle different schedule types correctly. The schedule implementation also included cron expression validation, date-time handling, and proper status updates based on user actions.

After the main task and schedule lifecycle flows were completed, visibility modules were added to improve monitoring and troubleshooting. Audit logs were implemented to track user actions and system changes. Troubleshoot flow was added to help users understand issues related to failed or problematic tasks. Execution history was also implemented to show past task runs, success or failure status, timestamps, and related execution details. These modules made the application more useful for operational users because they provided visibility into what happened in the system.

A major part of the development workflow was handling requirement changes without breaking the existing architecture. Since the project was reviewed continuously, several functional refinements were identified and incorporated during implementation. One important refinement was the replacement of temporary role toggles with an actual login-based role flow. This made the system more realistic and aligned it with enterprise access control expectations.

Another important refinement was the introduction of a first-time password setup path. This ensured that new users could securely set up their password before accessing the system. This also improved the authentication flow and made the application more complete from a user management perspective.

The audit and history modules were also refined during development. Action audit data and run history data were separated clearly because both serve different purposes. Audit data focuses on who performed an action and what change was made, while run history focuses on task execution details such as whether a task ran successfully or failed. Separating these two types of information made the system easier to understand and more accurate.

Role-based restrictions were further improved during review cycles. Retry and run actions were restricted for Viewer users because Viewers were intended to have read-only access. This ensured that users with monitoring access could observe system activity without accidentally triggering operational actions. Such restrictions helped maintain governance and reduced the chance of misuse.

Operational pages were also improved by filtering deleted entities. Deleted tasks or schedules were not shown in active operational views unless required for history or audit purposes. This helped keep the user interface clean and prevented confusion. Users could focus on active and relevant records instead of seeing outdated or deleted information in daily workflows.

Task status behavior was also updated based on schedule state transitions. For example, when schedules were paused, resumed, deleted, completed, or not yet created, the corresponding task status had to be displayed correctly. This status alignment was important because users depend on status indicators to understand the current condition of a task. Incorrect status display could lead to wrong decisions or unnecessary troubleshooting.

Schedule entry logic and cron validation behavior were improved during implementation. Since cron expressions can be difficult for users to understand, validation was added before submission to prevent incorrect schedules from being created. A natural-language schedule preview was also added to improve usability. This allowed users to understand the selected schedule in a simpler format instead of relying only on technical cron syntax.

Progress reviews were held regularly to validate not only feature completion but also the clarity of the user flow. The reviews focused on whether users could understand the screens, complete actions easily, and identify important information without confusion. In many cases, UI behavior was adjusted based on review observations. These adjustments included button alignment, table density, action positioning, spacing, and wording consistency.

User interface refinement was an important part of the workflow because AutoTasker is intended for frequent operational use. In such systems, even small usability issues can affect productivity. For example, unclear button placement, crowded tables, inconsistent labels, or confusing messages can slow down users during daily work. Therefore, the UI was improved continuously to make the system more practical and user-friendly.

During the hardening cycles, additional quality improvements were implemented. Raw HTTP messages were replaced with meaningful error toasts so that users could understand the problem clearly. Instead of showing technical backend errors, the application displayed simple and actionable messages wherever possible. This improved the overall user experience and made the system more suitable for non-technical users.

Confirmation dialogs were added before destructive actions such as deleting tasks or schedules. This helped prevent accidental changes and gave users a chance to review their action before proceeding. Since deletion and similar actions can affect operational workflows, confirmation dialogs added an extra layer of safety.

Status chips were made consistent across the application. These status indicators helped users quickly identify whether a task or schedule was active, paused, completed, failed, deleted, or not scheduled. Compact data presentation was also improved so that tables could display important information clearly without becoming too crowded.

Empty states and loading states were handled more carefully during the hardening phase. If no records were available, the application displayed a proper message instead of showing a blank page. If data was being fetched from the backend, loading indicators were displayed so that users understood that the system was processing the request. These improvements made the application feel more complete and reliable.

Duplicate request prevention was also implemented. Buttons connected to backend actions were disabled while requests were being processed. This prevented repeated clicks from creating duplicate tasks, repeated schedule changes, or multiple backend calls for the same action. This safeguard improved both frontend reliability and backend stability.

Throughout the development process, the workflow maintained a practical balance between backend-aligned correctness and user-friendly operation. On one side, the application had to follow backend rules, API contracts, status models, and permission behavior accurately. On the other side, the interface had to remain simple, understandable, and efficient for users. This balance was achieved through regular review cycles, disciplined incremental delivery, and continuous refinement of both functionality and user experience.

Overall, the iterative development workflow helped improve the quality and stability of the AutoTasker system. It allowed the project to evolve based on feedback while still maintaining a clean structure. By implementing features in smaller cycles, validating them regularly, and refining them based on real observations, the project moved from basic functionality toward a more reliable and enterprise-ready application.

2.5 Mid-Term and Final Review Presentations

The mid-term presentation was an important milestone in the project because it was used to demonstrate the first complete vertical flow of the AutoTasker system. At this stage, the project had moved beyond basic setup and individual screen development. The focus was on showing how the main parts of the application worked together as a connected flow. This included user login, role-based access, task creation, schedule creation, and basic task or schedule operations.

The main objective of the mid-term presentation was to validate the project direction before expanding into more advanced modules. The presentation highlighted the architecture approach, the user role model, and the core scheduler lifecycle. It showed how different users would access the application based on their role and how task and schedule operations would be controlled through the interface. This helped confirm whether the planned structure was suitable for an enterprise-oriented self-service scheduler utility.

During the mid-term review, special attention was given to the role model. Since AutoTasker involved multiple user types such as Viewer, Editor, and Admin, it was important to ensure that each role had the correct level of access. The review helped identify where action restrictions needed to be improved. For example, users with read-only access should not be able to perform operational actions such as running, retrying, editing, or deleting tasks. This feedback helped improve the permission handling in the second half of the project.

The review also helped identify improvements in user flow consistency. Some screens and actions needed better alignment so that users could move through the system more naturally. The positioning of buttons, naming of actions, and structure of tables were reviewed from a usability perspective. Since the system is intended for operational users, the interface had to be easy to understand and should not require detailed backend knowledge. This feedback helped improve the clarity and consistency of the user experience.

Scheduler usability was another major area discussed during the mid-term review. Since scheduling is the core function of the application, it was important that users could configure recurring and non-recurring schedules correctly. The feedback highlighted the need for better validation, clearer schedule entry logic, and easier interpretation of cron-based recurring schedules. This later resulted in improvements such as cron validation, schedule previews, and clearer date-time handling.

The feedback received during the mid-term presentation played an important role in guiding the second half of the implementation. After this review, the focus shifted from building only the core functionality to making the system more complete, stable, and operationally ready. The project was expanded to include stronger visibility, better governance, improved troubleshooting, and refined user experience.

After the mid-term review, audit and task history modules were strengthened. The audit module was improved to track user actions clearly, such as task creation, schedule updates, pause or resume actions, and deletion activities. This provided better traceability and helped users understand who performed an action and when it was performed. The task execution history module was also improved to show details of previous task runs, execution status, timestamps, and failure-related information.

The troubleshoot flow was refined to make it more useful for operational users. Instead of only showing technical failure information, the troubleshoot module was designed to help users understand the problem in a clearer way. This supported the main project goal of reducing dependency on technical teams for routine investigation. By providing better visibility into task failures and retry-related actions, users could respond faster to operational issues.

Role-based action boundaries were tightened after the mid-term review. This ensured that each user role could perform only the actions assigned to them. Viewers were restricted to monitoring and reading information, Editors were allowed to perform task and schedule operations, and Admins were given access to governance-related modules. These restrictions improved system control and reduced the possibility of accidental misuse.

Admin utilities were also completed during the second half of implementation. These included modules such as user management, task type management, and other governance-related controls. Admin features were kept separate from normal operational features so that system-level configuration could be handled only by authorized users. This improved the governance structure of the application.

A significant amount of effort was spent on improving the clarity of content in the user interface. Labels, button text, status messages, error messages, and confirmation messages were reviewed and refined. This was important because operational users should be able to understand the available actions quickly. The system should not force users to understand backend terminology or technical details. Therefore, the interface was made more direct, clear, and action-oriented.

The final presentation focused on demonstrating the end-to-end completeness of the AutoTasker system. It showed how a user can log in, navigate through the application based on their role, manage tasks, configure schedules, monitor executions, troubleshoot failures, and access audit information. The final presentation also demonstrated how admin users can manage governance-related modules and how export/import functionality supports environment movement.

The login and role-based navigation flow demonstrated that users are directed to appropriate screens based on their access level. This confirmed that the authentication and permission model was working correctly. The task and schedule lifecycle flow demonstrated how users can create, update, pause, resume, delete, and manage scheduled activities through the application.

Cron-driven recurring configuration was also demonstrated as part of the final review. The system supported recurring schedule creation with validation so that invalid expressions could be avoided before submission. The natural-language schedule preview made the schedule easier to understand for users who may not be familiar with cron syntax. This improved usability and reduced configuration mistakes.

Troubleshoot and retry operations were shown to demonstrate how users can respond to task execution problems. The system provided visibility into failures and allowed permitted users to take corrective actions where applicable. Audit and execution history visibility were also demonstrated to show how the system maintains traceability and supports operational review.

The final review also included admin governance modules. These modules showed how Admin users can manage users, task types, and other system-level controls. This confirmed that the application was not only useful for daily operations but also supported administrative control and governance.

The export and import flow was demonstrated as a utility for environment movement. This feature supported the movement of configuration-related data between environments and reduced manual effort during setup or migration. It added practical value to the system from an operational and deployment perspective.

The final review emphasized stability and usability outcomes, not just the number of features completed. The application was evaluated based on whether users could perform daily scheduler actions independently, whether role restrictions were properly enforced, whether audit and history information was visible, and whether the interface was understandable for non-technical users. This helped show that the project had moved from a basic implementation to a more complete and production-oriented solution.

The final outcome demonstrated that operational users can manage scheduler-related activities without depending heavily on IT teams for every routine change. At the same time, governance and traceability remain intact through role-based permissions, audit logs, history records, and admin controls. This directly supports the main objective of the

project, which is to reduce manual dependency, improve responsiveness, and provide better visibility in real operational workflows.

Overall, Chapter 2 represents the complete activity journey of the internship project, starting from onboarding and requirement understanding and ending with final delivery. The project evolved through structured planning, phased implementation, regular technical reviews, and continuous practical refinements. Each stage contributed to improving the functionality, reliability, usability, and governance of the AutoTasker system.

Through this process, the AutoTasker project developed into a complete and production-oriented implementation. The journey helped convert the initial idea of a self-service scheduler utility into a working application that supports role-based access, task and schedule lifecycle control, audit visibility, execution history, troubleshooting, admin governance, and environment migration support.

CHAPTER 3

3.1 User Roles and Access Model

AutoTasker was planned as a complete self-service scheduler platform that supports daily operational workflows with role-based control. The requirement analysis was done to ensure that the system is useful for real users, technically stable, and aligned with backend service behavior.

The project scope includes user authentication, task management, schedule management, audit visibility, execution history tracking, troubleshoot support, and admin governance modules. The system must support both recurring and non-recurring schedules while giving clear feedback to users during configuration and runtime operations.

The requirement analysis focused on five main goals:

1. Centralize all operational scheduler actions in one interface.
2. Enforce strict role boundaries for Viewer, Editor, and Admin.
3. Ensure reliable schedule configuration with validation.
4. Improve issue diagnosis using audit and task history.
5. Deliver a responsive and maintainable product-ready system.

The project requirements were broken down into functional and non-functional categories to make implementation and testing measurable. Functional requirements define what the system must do. Non-functional requirements define how well it must perform in terms of usability, reliability, security, and maintainability.

3.2 Functional Requirements

The following functional requirements were identified and implemented as part of AutoTasker.

Authentication and User Access

- FR1. The system shall allow login using email and password.
- FR2. The system shall support first-time password setup flow for newly created users.
- FR3. The system shall support password change for existing users.
- FR4. The system shall fetch and store user role level after successful login.

Role-Based Access Control

FR5. The system shall provide role levels as Viewer (0), Editor (1), and Admin (2).

FR6. Viewer users shall have read-only access to operational screens.

FR7. Editor users shall be able to create and modify tasks and schedules.

FR8. Admin users shall have full platform control including user and task type management.

FR9. Unauthorized routes and actions shall be restricted based on role level.

Access / Module	Viewer	Editor	Admin
View Dashboard (Admin → admin, User → viewer/editor)	✓	✓	✓
View Tasks (all visible tasks)	✓	✓	✓
View Task Details	✓	✓	✓
View Audit	✓	✓	✓
View Troubleshoot (error tasks)	✓	✓	✓
Run Task (manual retry/run now)	X	✓	✓
Create Task	X	✓	✓
Edit Task	X	✓	✓
Delete Task	X	✓	✓
Create / Edit / Delete Schedule	X	✓	✓
Pause / Resume Schedule	X	✓	✓
View Task History	✓	✓	✓
Manage Task Types	X	X	✓
Manage Users (create/delete/assign roles)	X	X	✓
Change Own Password	✓	✓	✓

Figure 4: User Role Access Matrix

User Management

FR10. Admin shall be able to add users with name, email, and role level.

FR11. Admin shall be able to list existing users.

FR12. Admin shall be able to delete users with confirmation checks.

FR13. User creation input shall validate correct email format.

Task Type Management

FR14. Admin shall be able to create task types.

FR15. Admin shall be able to update task type title and execution path as per backend rules.

FR16. Admin shall be able to delete task types.

FR17. Task type list shall be available for task creation and filtering.

Task Management

FR18. Editor/Admin shall be able to create tasks with required metadata.

FR19. The task form shall support name, description, type, and email list.

FR20. Editor/Admin shall be able to update task information.

FR21. Editor/Admin shall be able to logically delete tasks.

FR22. Deleted tasks shall not appear in active task listing pages.

FR23. Task status shall be displayed with consistent status indicators.

Schedule Management

FR24. Editor/Admin shall be able to add schedules to tasks.

FR25. The system shall support recurring schedules using cron expression.

FR26. The system shall support non-recurring schedules using date and time input.

FR27. Editor/Admin shall be able to edit schedule details.

FR28. Editor/Admin shall be able to pause and resume schedule execution states.

FR29. Editor/Admin shall be able to logically delete schedules.

FR30. Deleted schedules shall not appear in active schedule listing pages.

FR31. If no active schedule exists for a task, task status shall be shown as Not Scheduled.

Validation and Scheduling Safety

FR32. Cron expressions shall be validated before backend submission.

FR33. Invalid cron input shall return user-friendly error feedback.

FR34. Non-recurring schedules shall not allow past date-time configuration.

FR35. Schedule form shall show human-readable schedule preview from cron values.

Execution and Troubleshoot

FR36. Editor/Admin shall be able to trigger manual run action for a task.

FR37. Viewer shall not see or invoke restricted execution actions such as retry/run.

FR38. Troubleshoot module shall prioritize tasks currently in error state.

FR39. Troubleshoot module shall support retry action for authorized users.

Audit and Task History

FR40. The system shall provide audit records for user/system actions.

FR41. Audit rows shall include key context such as user, task, timestamp, action, and details.

FR42. The system shall provide task execution history records.

FR43. History shall include status, start time, finish time, exit code, and output snippet.

FR44. History and audit views shall support compact listing with expandable details.

Export and Import

FR45. Admin shall be able to export task configurations as JSON.

FR46. Admin shall be able to import task configuration JSON into target environment.

FR47. Export/import actions shall provide clear success or failure feedback.

User Experience and Action Safety

FR48. Destructive actions shall require confirmation dialog.

FR49. Buttons triggering backend operations shall disable during request processing.

FR50. All key flows shall include loading, empty, success, and error states.

3.3 Non-Functional Requirements

The following non-functional requirements were identified to ensure practical usability and production readiness.

Usability Requirements

NFR1. The interface shall be simple, readable, and role-oriented for daily operations.

NFR2. UI wording shall be clear and action-focused for non-technical users.

NFR3. Form validations shall provide immediate and understandable feedback.

NFR4. Data-heavy modules such as audit and history shall use compact display patterns.

Performance Requirements

NFR5. Page-level interactions shall remain responsive during common operations.

NFR6. API-driven actions shall provide visible loading indicators to users.

NFR7. Re-render overhead shall be controlled through modular component design.

Reliability Requirements

NFR8. Duplicate request submission shall be prevented through button-disable behavior.

NFR9. Invalid data shall be blocked at UI level before backend submission where possible.

NFR10. Critical status transitions shall remain consistent across list and detail views.

Security and Governance Requirements

NFR11. Route and action visibility shall be role restricted.

NFR12. Sensitive admin operations shall be shielded from lower roles.

NFR13. Authentication state shall be checked before allowing protected modules.

NFR14. Error handling shall avoid exposing confidential backend internals in UI messages.

Maintainability Requirements

NFR15. Codebase shall follow modular feature-based structure.

NFR16. API contracts shall be handled through typed models.

NFR17. Shared UI components shall be reusable across screens.

NFR18. Business rules shall be centralized where possible to minimize duplication.

Scalability Requirements

NFR19. Architecture shall support new modules without major rewrite.

NFR20. Role matrix shall be extendable for future permission models.

NFR21. Scheduling support shall remain extensible for additional recurrence patterns.

Compatibility Requirements

NFR22. The application shall run in standard modern browser environments.

NFR23. Environment-based API base URL configuration shall support multiple deployments.

NFR24. Timezone behavior shall remain aligned to backend scheduler expectations.

3.4 Hardware Requirements

The project does not require specialized hardware and can be developed and used on standard modern systems. However, minimum practical hardware specifications were defined for smooth development and testing.

Development Environment (Recommended)

1. Processor: Intel i5/Ryzen 5 or above.
2. RAM: Minimum 8 GB, recommended 16 GB for parallel tools.
3. Storage: Minimum 10 GB free space for project, dependencies, and build artifacts.
4. Display: 1366x768 minimum, Full HD recommended for dashboard review.
5. Network: Stable internet for dependency installs and remote repository access.

Execution/User Environment (Recommended)

1. Standard desktop or laptop capable of running a modern browser.
2. Minimum 4 GB RAM for smooth multi-tab operation.
3. Reliable network connectivity for API communication.

Server-Side (Backend Hosting Context)

1. API service host reachable from frontend environment.
2. Persistent storage support for users, tasks, schedules, audits, and history.
3. Scheduler execution environment integrated with backend run and status services.

3.5 Software Requirements

The software stack was selected to provide maintainability, usability, and clean backend integration.

Frontend Stack

1. React for component-based UI implementation.
2. TypeScript for static typing and safer development.
3. React Router for route-level module separation and guards.
4. Material UI for consistent and responsive component design.
5. Vite for fast local development and optimized production builds.

Backend Integration

1. REST APIs for auth, user management, task types, tasks, schedules, audits, history, and export/import.
2. Environment-based API URL configuration.
3. Typed request/response transformation layer in frontend.

Data and Utility Layer

1. Domain model definitions for role, task, schedule, and history state mapping.
2. Cron expression validation and schedule preview utility.
3. Date-time and status conversion utilities aligned to backend behavior.

Development and Delivery Tools

1. Node.js runtime and npm for dependency and script management.
2. Git for version control and incremental change tracking.

3. Browser developer tools for API and UI debugging.

Testing-Oriented Software Needs

1. API endpoint accessibility in development/test environment.
2. Support for role-based login accounts for Viewer, Editor, and Admin flow validation.
3. JSON file handling capability for export/import testing.

Summary of Requirement Analysis

The requirement analysis confirms that AutoTasker is not a limited scheduling page but a complete operational platform with governance, visibility, and action control. The defined functional and non-functional requirements provided a clear basis for architecture design, implementation, and validation. These requirements also ensured that the final system remains practical for enterprise use while being maintainable and extensible for future improvements.

CHAPTER 4

4.1 System Overview

AutoTasker is designed as a role-based task scheduling platform that provides centralized control over operational jobs. The core design goal is to give users a single, reliable interface for creating tasks, configuring schedules, monitoring execution, troubleshooting failures, and governing access without exposing internal implementation details.

The system follows a layered and modular structure where each module has a clear responsibility. User-facing workflows are handled in the web application, business operations are executed through backend APIs, and state persistence is handled through structured data entities such as users, task types, tasks, schedules, audits, and execution history.

At a high level, the platform supports:

- Authentication and identity flow.
- Role-aware route and action control.
- Task and schedule lifecycle management.
- Action-level audit trail.
- Execution-level history analysis.
- Admin governance and environment migration operations.

The design emphasizes operational clarity. Instead of presenting raw technical controls, it provides guided interactions such as cron field inputs, schedule preview, confirmations for sensitive actions, and meaningful status visualization. This reduces user mistakes while preserving flexibility for advanced scheduling needs.

4.2 System Architecture

The architecture of AutoTasker is designed in a multi-layer format to separate presentation, business interaction, and data operations.

Layer 1: Presentation Layer (Frontend UI)

- Built with React and TypeScript.
- Contains pages, forms, tables, dialogs, and role-based navigation.
- Responsible for user interactions, validations, and display logic.

Layer 2: Application Control Layer

- Contains routing, access gating, and module-level workflow control.
- Decides what actions are visible based on user role.
- Coordinates screen-level state changes after API operations.

Layer 3: API Integration Layer

- Centralized client utilities for backend communication.
- Typed request/response mapping to avoid inconsistent data handling.
- Error transformation from backend responses to user-friendly messages.

Layer 4: Backend Service Layer

- Exposes endpoints for auth, users, task types, tasks, schedules, audits, task history, and export/import.
- Applies server-side business rules, role validation, status handling, and execution interaction.

Layer 5: Data Persistence Layer

- Stores domain entities and operational records.
- Keeps audit events and execution history separately for better traceability.
- This layered architecture allows each concern to evolve independently. For example, UI changes can be made without modifying persistence design, while backend status rules can be updated without redesigning screen structure if contracts remain consistent.

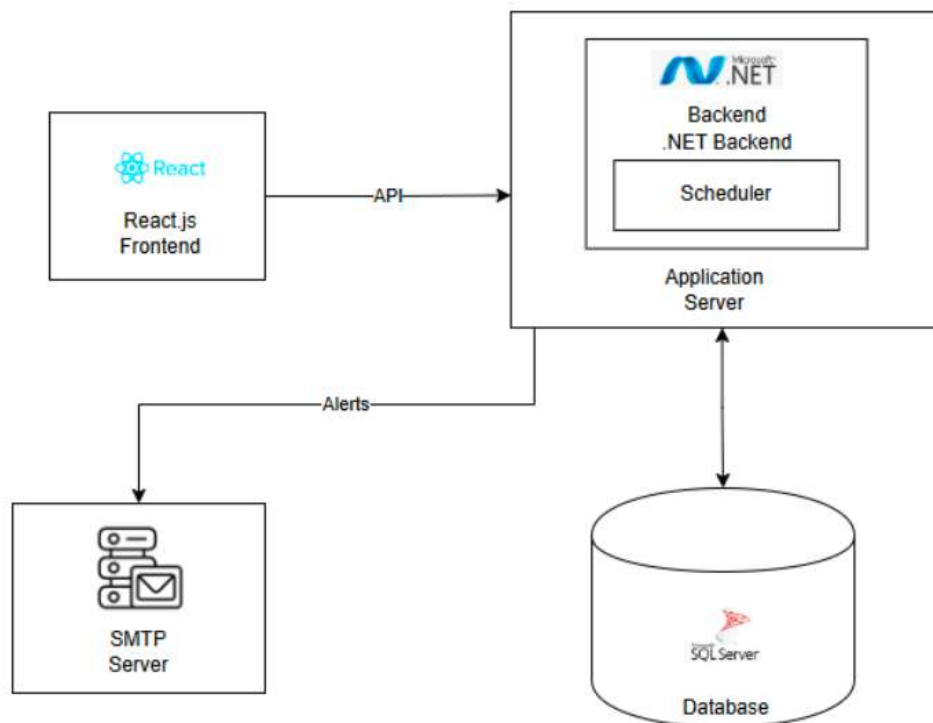


Figure 5: Tech Stack and System Architecture Diagram

4.3 Module Description

The complete platform is divided into functional modules.

4.3.1 Authentication Module

Purpose:

Login, initial password setup, and password update.

Key design points:

- Email-first login flow to identify user state.
- First-time users are redirected to set password screen.
- Role level from login response drives all access behavior.

4.3.2 Role and Access Module

Purpose:

Enforce role-based restrictions across navigation and actions.

Key design points:

- Viewer has read-only access.
- Editor has operational mutation access.
- Admin has governance and full platform access.
- Access checks applied at both route level and action level.

4.3.3 User Management Module (Admin)

Purpose:

Add, view, and delete users.

Key design points:

- Email format validation at input stage.
- Confirmation modal before user creation.
- Safe delete flow with backend-aware conflict handling.

4.3.4 Task Type Management Module (Admin)

Purpose:

Manage task categories and execution path metadata.

Key design points:

- Create, update, and delete operations.
- Constraints aligned with backend rules when task type is in use.
- List exposed to task creation workflows.

4.3.5 Task Management Module**Purpose:**

Manage complete task lifecycle.

Key design points:

Create task with name, description, type, and email list.

Edit task metadata.

Manual run support for authorized users.

Logical deletion with visibility filtering in UI.

Status display with consistent state chips.

4.3.6 Schedule Management Module**Purpose:**

Attach and control execution schedules under tasks.

Key design points:

- Supports recurring and non-recurring modes.
- Recurring mode uses cron expression with structured input and manual override.
- Non-recurring mode uses date and time entry with past-time validation.
- Pause/resume and logical delete controls for schedules.
- Task status derived from schedule-state composition.

4.3.7 Audit Module**Purpose:**

Show who performed which action and when.

Key design points:

- Compact view for high-volume readability.
- Action metadata shown in table.
- Expandable detail body for deeper inspection.

4.3.8 Task History Module

Purpose:

Show actual run outcomes independent of action logs.

Key design points:

- Includes status, start/end timestamps, exit code, and output snippet.
- Supports filtering to locate specific executions quickly.
- Useful for failure analysis and trend review.

4.3.9 Troubleshoot Module

Purpose:

Focus operational attention on error-state tasks.

Key design points:

- Lists tasks needing intervention.
- Retry action for authorized users.
- Quick navigation to task details for deeper inspection.

4.3.10 Export/Import Module (Admin)

Purpose:

Move task configurations between environments.

Key design points:

- Export JSON generation and download support.
- Import JSON upload and submit flow.
- Clear feedback for success and failure states.

4.4 Data Model and Status Lifecycle

The data model defines the main operational entities used by AutoTasker and their relationships. Users represent authenticated platform members, Task Types define reusable execution categories, Tasks store operational job metadata, Schedules define run timing, Audits record action-level traceability, and Task History records execution outcomes. The ER diagram is used here to replace a long textual listing of tables and keys with a clearer database-level view of primary keys, foreign keys, and entity relationships.

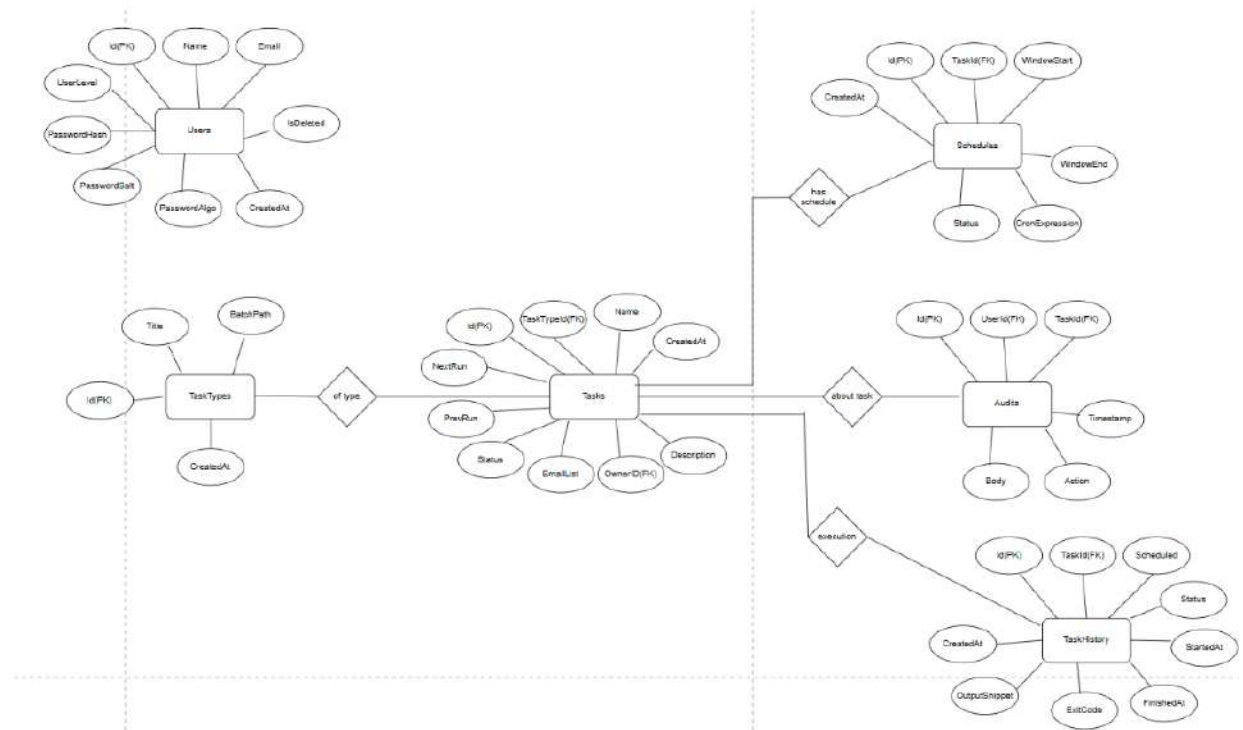


Figure 6: Backend ER Diagram

The status lifecycle is derived from these relationships. A task can exist without an active schedule, can be linked to one or more schedules, and can generate multiple audit and history records during its lifetime. This separation helps the system distinguish between configuration changes and actual execution results.

4.5 Use Case and Workflow Analysis

The use case diagram summarizes platform behaviour across Viewer, Editor, and Admin roles. Instead of listing every use case in repeated text, the diagram presents the complete operational boundary: task management, scheduling, troubleshooting, audits, export/import, user management, and task type management. This makes the section more concise while still showing role-based responsibilities and system-level interactions.

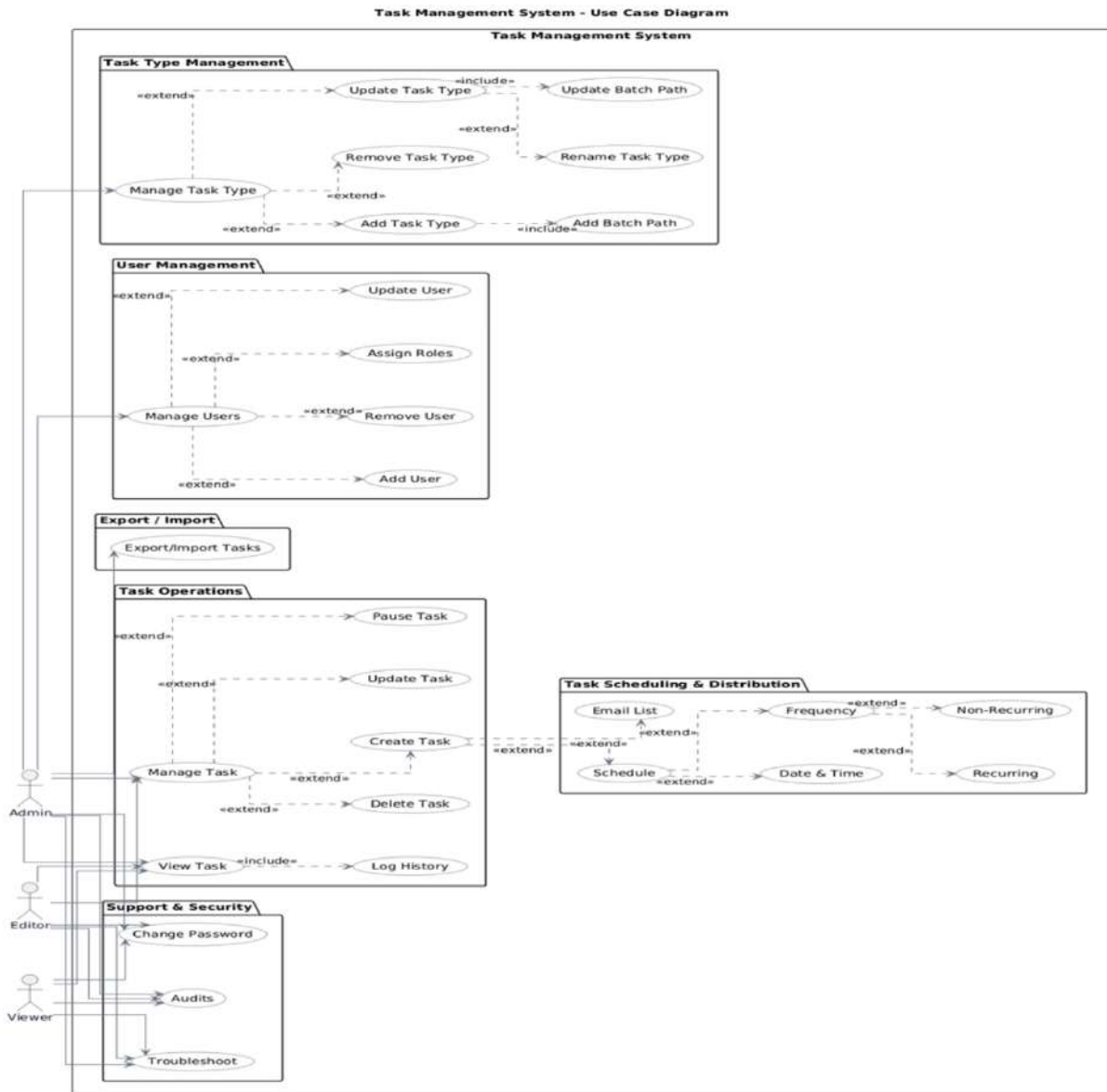


Figure 7: Use Case Diagram

Viewer users are limited to monitoring, audit review, troubleshoot visibility, and task history. Editor users can perform operational task and schedule actions such as create, update, run, pause, resume, and delete. Admin users have full governance access, including user management, task type management, and configuration export/import. The workflow also ensures that every sensitive mutation is connected to confirmation, validation, and traceability through audit and history records.

Summary of Chapter 4

The system design of AutoTasker provides a structured, scalable, and governance-aware foundation for scheduler operations. The architecture clearly separates responsibilities, the data model maps entities and operational logs, and the use case diagram confirms that the platform supports both day-to-day usability and controlled enterprise execution needs.

CHAPTER 5

5.1 Authentication and First-Time Password Setup

The authentication module is an important part of the AutoTasker: Self-Service Task Scheduler because the system is intended to be used by different categories of users with different levels of access. The application must ensure that business users can access only the task and schedule management features required for daily operations, while administrators can access monitoring and governance-related features. In the current frontend implementation, authentication is simulated using a React Context named `AuthContext`.

The `AuthContext` maintains the currently selected user role and provides a `switchRole()` function. This role-switching mechanism allows the frontend interface to behave either as a Business User or as an Admin. During development, this approach is useful because both user flows can be tested without depending on a completed backend authentication service. It also allows developers to verify whether role-based navigation, dashboard visibility, and restricted access messages are working correctly.

In the current implementation, the role value is consumed by different components and pages to decide what the user can view or perform. For example, a business user is expected to access the Client Dashboard, where task creation, schedule creation, pause, resume, and delete operations are available. An admin user is expected to access the Admin Dashboard, where monitoring data, operational logs, task health, and schedule status information are shown. This role separation gives the project a strong foundation for future authorization logic.

In a production implementation, the simulated role selection would be replaced by a secure login mechanism. The user would enter their credentials on a login screen, and the backend would validate the username and password. After successful validation, the backend would return a secure session token or JSON Web Token. This token would contain user identity details and role-based claims such as business or admin. The frontend would then store the token securely and use the role claims to control navigation and access.

The first-time password setup flow is another important feature for enterprise applications. When a new user account is created by an administrator, the user may initially receive a temporary password. During the first login, the backend can identify that the account is still in a temporary-password or first-login-pending state. The frontend would then redirect the user to a password setup page before allowing access to the dashboard.

On the first-time password setup screen, the user would be required to enter a new password and confirm it. The frontend would validate the password using rules such as minimum length, uppercase letter, lowercase letter, number, special character, and confirmation matching. These checks can be implemented using the same validation

approach already used in the project, such as Zod-based schema validation. If the password satisfies all rules, the frontend would send the request to the backend for final validation and update.

After successful password setup, the backend would update the user account status from first login pending to active. The user would then be redirected to the correct dashboard based on their role. Admin users would be routed to the Admin Dashboard, while business users would be routed to the Client Dashboard. This ensures that authentication, password initialization, and role-based authorization work together as one controlled access flow.

Although the current implementation uses a simulated authentication mechanism, the frontend architecture already supports future backend integration. Since role information is centrally managed through context, the current `switchRole()` logic can later be replaced with actual login claims without major changes to the dashboard and routing structure.

5.2 Role-Based Dashboards and Navigation

Role-based dashboards are implemented to provide separate experiences for business users and administrators. The application uses `react-router-dom` to manage page navigation. The main routes include `/client` for the Client Dashboard and `/admin` for the Admin Dashboard. The root route redirects the user to the client dashboard by default. This routing structure helps organize the project into clear functional areas.

The common layout of the application is handled through the `AppShell` component. This component contains the header, navigation links, role selector, and main content area. By placing common layout elements in a reusable shell, the application avoids repeated layout code across pages. It also ensures that navigation remains consistent throughout the system.

The Client Dashboard is designed mainly for business users. It allows users to create and manage scheduled tasks. The dashboard includes forms for creating tasks and schedules, tables for viewing existing records, status indicators, confirmation dialogs, and troubleshooting logs. Business users can perform operational actions such as creating tasks, creating schedules, pausing tasks, resuming tasks, and deleting records based on permitted access.

The Admin Dashboard is designed for administrative monitoring. In the current implementation, the admin area focuses more on visibility and operational awareness rather than direct modification of task data. It displays monitoring cards, task health information, schedule status, runtime activity, and logs. This allows administrators to understand the overall condition of the scheduling system without interfering with business-user operations.

Access restriction is implemented at the dashboard level. When a user without the admin role tries to access the admin route, the application displays a restricted access message

instead of showing the admin content. This ensures that administrative views are not exposed to normal business users. Although this is currently handled at the frontend level, in a real production system, the backend must also enforce authorization checks to prevent unauthorized access through direct API calls.

The role-based structure also improves maintainability. Since the application already separates business and admin areas, future features can be added to the correct dashboard without affecting unrelated modules. For example, user management and task type management can be added to the admin section, while task creation and schedule creation can remain in the client section.

When real authentication is introduced, the current role toggle can be removed. The role can instead be populated from the authenticated user's token or profile response. This means the current routing and conditional rendering logic can continue to be reused with minimal changes. Therefore, the current implementation acts as both a working frontend prototype and a scalable base for production authorization.

5.3 Task and Schedule Management

Task and schedule management is the core functionality of the AutoTasker: Self-Service Task Scheduler. The main objective of this module is to allow users to create scheduled tasks without depending completely on technical teams or manual backend configuration. The implemented frontend provides forms, validation, table views, action buttons, confirmations, and user feedback to support this workflow.

The TaskForm component is used to create a new task. The form captures important task details such as task name, description, task type, owner, and batch file name. The task type field supports values such as T1, T2, T3, and T4, which are defined in the project's domain model. The batch file name field is used to map the scheduled task to the batch file or execution unit that will eventually be triggered by the scheduler.

Form validation is implemented using Zod. This ensures that incomplete or invalid data is not submitted. For example, required fields must be filled before the task can be created. Validation at the frontend level improves user experience because errors are shown immediately without waiting for a backend response. In a production system, the backend should also repeat the same validation rules to ensure data integrity and security.

When the user submits a valid task, the Client Dashboard calls the createTask mutation from the useSpectrumData hook. This mutation passes the task data to the mock API. The mock API then creates a new task object, assigns an auto-generated task ID, sets the task status to active, records timestamp values, and adds the task to the mocked task list. A log entry is also generated to indicate that the task was created successfully.

Schedule creation is handled through the ScheduleForm component. A schedule must be associated with an existing task. This prevents orphan schedules from being created without a parent task. The form allows the user to select the task, run time, frequency,

schedule mode, timezone, and enabled status. Supported frequencies include hourly, daily, weekly, and monthly. Supported schedule modes include recurring and non-recurring.

The schedule form is designed to be simple and business-user friendly. Instead of asking users to write technical cron expressions, the form captures scheduling details through readable fields. This reduces user error and makes the system easier to use for non-technical users. When no task exists, the schedule creation form is disabled or blocked, ensuring that only valid schedules can be created.

Task lifecycle actions are also implemented. Users can pause and resume tasks. Pausing a task temporarily prevents it from being treated as active, while resuming a task restores it to active status. These actions are useful when a task should not be deleted permanently but needs to be stopped for a short period due to maintenance, dependency issues, or business requirements.

Deletion behavior is implemented carefully. When a task is deleted, all schedules linked to that task are also removed. This follows a cascade delete pattern. The application communicates this behavior clearly to the user through confirmation dialogs and table descriptions. This prevents confusion and ensures that users understand the impact of deleting a task. Deleting a schedule, however, removes only the selected schedule and keeps the parent task intact.

The use of confirmation dialogs improves safety. Since actions such as deleting a task can affect related schedules, the user is asked to confirm before the action is completed. Toast notifications are used to provide immediate feedback after actions such as task creation, schedule creation, deletion, pause, and resume. This makes the system more interactive and user-friendly.

Overall, the task and schedule management module successfully demonstrates the main operational flow of the project. It allows users to create tasks, assign schedules, control task status, delete records safely, and view the result of each action through tables and logs.

5.4 Cron Builder and Natural Language Preview

The current implementation provides a schedule creation interface using simple input fields such as run time, frequency, mode, timezone, and enabled status. This design is suitable for business users because it hides the complexity of cron syntax. Users can create schedules by selecting understandable options rather than manually entering technical scheduler expressions.

A cron builder module can be added on top of the existing schedule form. The values selected by the user can be converted into a cron expression before being sent to the backend scheduler. For example, if the user selects a daily schedule at 09:00 UTC, the frontend or backend can generate the corresponding cron expression for execution every

day at 09:00. Similarly, hourly, weekly, and monthly schedules can be converted into the appropriate cron patterns.

For weekly schedules, additional controls can be introduced to select the day of the week. For monthly schedules, the form can include a day-of-month selector. These controls would make the schedule builder more flexible while still keeping it user-friendly. The generated cron expression can be stored by the backend along with the original schedule metadata.

The natural language preview is an important usability feature. Instead of displaying only a cron expression, the interface can show a readable explanation of the schedule. For example, the preview may display sentences such as Runs every day at 09:00 UTC, Runs every Monday at 16:30 Asia/Kolkata time, or Runs on the 1st day of every month at 10:00 Europe/London time. This helps users verify their configuration before saving it.

This feature is especially useful because cron syntax can be difficult for non-technical users to understand. A natural language preview reduces the chances of incorrect scheduling. It also increases user confidence because they can clearly see when the task will run.

The current domain model already supports this extension. Each schedule stores fields such as runAt, frequency, mode, timezone, and enabled. These fields can be used to generate both cron expressions and natural language summaries. Therefore, the cron builder can be implemented without major changes to the existing schedule structure.

In the production version, the backend scheduler should perform the final conversion and validation. The frontend can display a preview, but the backend should confirm whether the generated schedule is valid and supported. This ensures that the scheduler behavior remains consistent and reliable across environments.

5.5 Audit, Troubleshoot, and Task History

Audit and troubleshooting features are necessary for any scheduled-task system because users and administrators need to understand what actions were performed and what happened during task execution. The current implementation includes a LogViewer component that displays operational logs. These logs simulate how a real backend logging or audit service would behave.

Each log entry contains useful information such as task name, timestamp, severity level, message, and source. The supported severity levels are INFO, WARN, and ERROR. The log source can be task-manager, scheduler, or batch-runner. These categories help users identify where a particular event or issue occurred.

The LogViewer includes a search box and severity filter. The search feature allows users to search logs by task name or message content. The severity filter allows users to view

only specific types of logs, such as warnings or errors. This is useful when troubleshooting failed executions, skipped schedules, paused tasks, or system issues.

The mock API applies filters before returning log data, which simulates a backend log filtering endpoint. In a production system, these filters would be applied at the database or logging service level. This would allow the frontend to retrieve only the most relevant log records instead of loading all logs at once.

Task history is simulated through log generation. When important actions occur, such as task creation, task deletion, task pause, task resume, schedule creation, or schedule deletion, the mock API automatically adds a log entry. This creates a basic audit trail. For example, when a task is deleted, a warning log is generated to indicate that the task and its linked schedules were removed.

In a production implementation, audit records should be stored persistently. Each audit record should contain the user ID, role, action type, task ID, schedule ID if applicable, previous state, new state, timestamp, and request source. For security and compliance purposes, additional information such as IP address or browser session details can also be stored where required.

Audit history helps administrators track user activity. It can answer questions such as who created a task, who paused a schedule, when a task was deleted, and what changes were made. This improves accountability and makes the system more suitable for enterprise environments.

Troubleshooting logs also help reduce dependency on backend teams. Business users and administrators can inspect basic failure reasons directly from the interface. For example, if a task fails because a batch file is missing or a schedule is disabled, the log viewer can show this information clearly. This supports faster issue resolution and improves operational transparency.

5.6 Admin Modules: Manage Users and Task Types

The current Admin Dashboard focuses on monitoring and visibility. It includes status cards, task health information, schedule tables, runtime activity, and log monitoring. This gives administrators a central view of the system's operational condition. However, user administration and task type management are not fully implemented in the current frontend code. These can be added as implementation-ready modules within the same admin architecture.

The Manage Users module would allow administrators to control user access. Admins should be able to create new users, assign roles, reset passwords, activate accounts, deactivate accounts, and mark users for first-time password setup. Each user record can include fields such as name, email address, role, account status, created date, and last login timestamp.

This module would directly support the authentication flow. For example, when an admin creates a new user, the account can be created with a temporary password or first-login-pending status. When the user logs in for the first time, the system can redirect them to the first-time password setup page. After the password is changed successfully, the account status can be updated to active.

Role assignment is another key function of user management. If a user is assigned the business role, they should be able to access the Client Dashboard. If a user is assigned the admin role, they should be able to access the Admin Dashboard. This ensures that system access is controlled by administrator-defined permissions rather than hardcoded frontend logic.

The Manage Task Types module would allow administrators to maintain task categories such as T1, T2, T3, and T4. In the current implementation, these task types are hardcoded in the frontend domain model and task form. While this is acceptable for a prototype, a production system should fetch task types from the backend.

By managing task types from the admin panel, administrators can add new task types, update task type descriptions, disable unused task types, and control which task types are available to business users. This makes the system more flexible and avoids the need to modify frontend code whenever a task category changes.

Each task type can include fields such as task type code, description, status, created date, and updated date. If a task type is disabled, it should no longer appear in the task creation form. However, existing tasks using that type can still be shown for history and traceability.

Adding these admin modules would make the system more complete. Business users would focus only on operational task and schedule management, while administrators would manage users, permissions, task types, and system configuration. This separation improves governance and maintainability.

5.7 Export and Import for Environment Migration

Export and import functionality is useful when task configurations need to be moved between environments such as development, testing, staging, and production. In many enterprise systems, scheduled tasks are first configured and tested in a lower environment before being moved to production. Manually recreating each task and schedule in every environment can be time-consuming and error-prone.

The current implementation stores task and schedule data in mocked in-memory arrays. However, the domain model is already structured in a way that supports serialization. Tasks, schedules, task types, and configuration metadata can be converted into a structured file format such as JSON.

The export module would collect selected task and schedule data and generate a downloadable file. Each exported task can include task name, description, task type, owner, batch file mapping, status, created timestamp, and updated timestamp. Each related schedule can include schedule ID, task ID, run time, frequency, mode, timezone, enabled status, and created timestamp.

The export feature can also support selective export. For example, the user may choose to export all tasks, only active tasks, or only tasks belonging to a particular owner or task type. This gives administrators more control during environment migration.

The import module would allow users to upload an exported configuration file. Before applying the imported records, the system should validate the structure and content of the file. Validation is necessary to prevent invalid or unsafe data from entering the target environment.

Important validation checks include duplicate task names, missing task fields, unsupported task types, invalid timezones, missing batch file names, schedules without parent tasks, and invalid frequency values. The import screen should show a preview of the records before applying them. This preview can display which records will be created, skipped, updated, or rejected.

For safety, the backend should perform the final import operation as a transaction. If a critical validation error occurs, the system should either stop the complete import or continue only with valid records based on the selected import mode. This prevents partial and inconsistent migration.

The import module should also generate audit logs. These logs should record who performed the import, when it was done, how many records were imported, how many failed, and what environment the import was applied to. This is important for traceability and rollback planning.

Export and import functionality would reduce manual setup effort, improve consistency between environments, and make the system more suitable for real enterprise deployment.

CHAPTER 6

6.1 Role-Based Flow Validation

The role-based flow of the AutoTasker: Self-Service Task Scheduler was validated by testing the application in both Business User and Admin modes. Since the current implementation is a frontend prototype, the role-based behavior is simulated using AuthContext. This context stores the active role of the user and allows switching between business and admin roles. This made it possible to test different user experiences without requiring a completed backend authentication system.

The first part of validation focused on the Business User role. When the active role was set as business, the user was able to access the Client Dashboard. The Client Dashboard provided all major operational features required by a normal business user. These included task creation, schedule creation, pause and resume actions, deletion of schedules, deletion of tasks, and access to troubleshoot logs. The interface was designed to support self-service task management, so the business user could configure scheduled tasks without depending on administrators or backend teams for every change.

The Business User flow was reviewed by performing task-related actions from the Client Dashboard. A user could create a new task by entering required information such as task name, description, task type, owner, and batch file name. After creating a task, the user could create a schedule for that task. The user could also pause or resume the task depending on the current status. These operations confirmed that the business user interface was not only view-based but also supported the expected operational actions.

The second part of validation focused on the Admin role. When the active role was changed to admin, the Admin Dashboard became accessible. The Admin Dashboard provided monitoring-focused information rather than direct task editing. It displayed important metrics such as total tasks, active tasks, paused tasks, error tasks, runs today, failure rate, runtime activity, task list, schedule list, and logs. These details helped validate that the administrator view was designed mainly for supervision and operational visibility.

The Admin Dashboard was reviewed to ensure that it provided a broader system-level view compared to the Client Dashboard. While the Client Dashboard focused on creating and maintaining individual tasks and schedules, the Admin Dashboard focused on understanding the overall health of the scheduled-task system. This separation is important because administrators usually need to monitor system performance, identify failures, and review execution activity rather than perform daily business-user task creation.

Access restriction was also tested. When a non-admin user attempted to open the Admin Dashboard, the application displayed a restricted access message instead of showing the admin content. This confirmed that the frontend correctly separated normal user functionality from administrator functionality. Although the restriction is currently

implemented at the frontend level, it demonstrates the expected role-based behavior for the final system.

The role-based flow validation confirmed that the application provides different views based on user responsibility. Business users are given access to task and schedule management features, while administrators are given access to monitoring and review features. This satisfies the basic requirement of role-based access and prepares the application for future integration with real authentication and backend authorization.

6.2 Task Lifecycle and Schedule Validation

The task lifecycle was tested from the point of task creation to task deletion. This validation was important because task management is the core feature of the AutoTasker: Self-Service Task Scheduler. The objective was to verify whether a task could be created, displayed, updated through status actions, linked with schedules, and deleted safely.

Task creation was tested using the task form available in the Client Dashboard. The user entered details such as task name, description, task type, owner, and batch file name. The supported task types were T1, T2, T3, and T4. These task types represent the categories available in the current domain model. The batch file name field was used to associate the task with the batch execution file that would be triggered by the scheduler in a production implementation.

Validation was applied before submitting the task form. The project uses Zod validation to prevent incomplete or invalid task data from being submitted. Required fields had to be filled correctly before the task could be created. This helped ensure that the task table did not receive incomplete records. The validation also improved user experience because users could immediately identify missing or incorrect fields.

After a task was successfully created, the new task appeared in the task table. The table displayed important task details such as task name, description, task type, status, owner, and batch file name. The task status was set to active by default. This confirmed that the mock API correctly created the task object, assigned its status, and updated the displayed task list.

The pause and resume behavior was also tested. When an active task was paused, its status changed from active to paused. When the same task was resumed, its status returned from paused to active. This confirmed that the application correctly handled task state changes. This feature is useful in real operational scenarios where a scheduled task may need to be temporarily stopped due to maintenance, dependency issues, or business requirements.

Schedule validation was tested after creating at least one task. The schedule form required a task to be selected before a schedule could be created. This ensured that schedules were always linked to an existing parent task. The schedule form collected details such as task, run time, frequency, mode, timezone, and enabled status. Supported

frequencies included hourly, daily, weekly, and monthly. Supported modes included recurring and non-recurring schedules.

The application prevented schedule creation when no task existed. This confirmed that the system avoided orphan schedules. An orphan schedule is a schedule that does not belong to any valid task, which can create confusion and execution errors in a scheduler system. By allowing schedule creation only after task creation, the frontend maintained the correct parent-child relationship between tasks and schedules.

The delete behavior was validated separately for tasks and schedules. When a schedule was deleted, only that selected schedule was removed from the schedule list. The parent task remained available in the task table. This confirmed that schedule deletion did not affect the task itself.

Task deletion was tested separately because it has a larger impact. When a task was deleted, all schedules connected to that task were also removed. This cascade deletion behavior matched the expected relationship between tasks and schedules. Since schedules depend on tasks, it is logical that deleting a task should remove the schedules associated with it. The confirmation dialog also warned the user before deletion, reducing the chance of accidental removal.

Overall, the task lifecycle and schedule validation confirmed that the application handled the main operational rules correctly. Tasks could be created, viewed, paused, resumed, and deleted. Schedules could be created only for existing tasks and removed independently. Task deletion correctly removed linked schedules. These results show that the implemented frontend supports the expected task and schedule management workflow.

6.3 Audit and Task History Output Screens

Audit and task history behavior was reviewed through the Troubleshoot Logs section of the application. This section displays logs generated from mocked backend events. Even though the current logs are not stored in a real database, they represent the type of operational history that would be expected in a production scheduled-task system.

The logs were generated for important task and schedule activities. These activities included task creation, schedule creation, pause and resume actions, task deletion, schedule deletion, skipped task execution, retry events, and failed batch execution. This helped validate that the application could show a meaningful history of system activity instead of only displaying the current state of tasks and schedules.

Each log entry displayed important information such as severity level, task name, message, timestamp, and source. The severity level helped identify whether the log was informational, warning-based, or error-based. The supported levels were INFO, WARN, and ERROR. This made it easier to separate normal activities from potential issues.

The log source field provided additional clarity. The source could be task-manager, scheduler, or batch-runner. The task-manager source represented task configuration and lifecycle actions. The scheduler source represented scheduling-related events such as skipped or triggered runs. The batch-runner source represented actual batch execution behavior. This separation helped users understand where an issue originated.

The search feature in the log viewer was tested by searching for task names and message content. This allowed specific records to be found quickly, especially when many logs were displayed. For example, if a particular task had failed, the user could search for the task name and view only the related logs. This made troubleshooting more focused and reduced the need to manually inspect every log entry.

The severity filter was also tested. The user could filter logs by INFO, WARN, ERROR, or all levels. This was useful for reviewing only important issues. For example, ERROR logs could be selected to view failed batch executions, while WARN logs could be used to review skipped tasks, deleted schedules, or cautionary events. INFO logs helped show normal successful actions such as task creation or schedule creation.

The audit output showed that the frontend provides a useful operational history for review and troubleshooting. It allows users to understand what action happened, when it happened, and which part of the system generated the event. This is important for a scheduled-task system because users may need to identify why a task did not run, why a schedule was removed, or when a task was paused.

Although the current implementation uses mocked logs, the same screen can be connected to backend audit tables, log APIs, or centralized logging platforms in the future. In a production version, each audit record should include more details such as user ID, role, action type, previous state, new state, task ID, schedule ID, timestamp, and request source. This would improve traceability and make the system suitable for enterprise use.

The review of audit and task history screens confirmed that the project supports the basic visibility needed for troubleshooting. The logs provide context for user actions and system events. This reduces dependency on backend teams and helps operational users understand task behavior directly from the interface.

6.4 UI/UX Improvements from Review Feedback

Several user interface and user experience improvements were included in the application to make it easier to use, review, and demonstrate. Since the system is intended for operational users, the interface needed to be simple, clear, and informative. The improvements focused on layout organization, form feedback, action safety, status visibility, and responsiveness.

The application uses a clean dashboard layout with separate areas for each major function. The Client Dashboard contains separate panels for task creation, schedule

creation, task table, schedule table, and troubleshoot logs. This separation makes the page easier to understand because each operation has its own defined space. Users can quickly identify where to create a task, where to create a schedule, and where to view existing records.

The Admin Dashboard was also structured for quick understanding. It includes status cards, task health information, runtime activity, task list, schedule list, and logs. The status cards provide a quick summary of the system, including total tasks, active tasks, paused tasks, error tasks, runs today, and failure rate. This helps administrators understand the system condition without reading every row in the task table.

Form validation feedback was added to improve data entry. When users leave required fields empty or enter invalid data, validation messages are shown. This helps users correct mistakes before submitting the form. Immediate validation feedback is useful because it prevents confusion and reduces failed submissions.

Loading states were included for actions such as task creation and schedule creation. For example, buttons can show text such as Creating while the operation is in progress. This prevents repeated clicks and informs the user that the application is processing the request. This improves the reliability and smoothness of the user experience.

Toast notifications were included to provide feedback after actions. When a task is created, a schedule is created, a task is paused, a task is resumed, or a record is deleted, the user receives a notification. These notifications confirm whether the action was successful or whether an error occurred. This is important because users should not have to guess whether their action was completed.

A confirmation dialog was added before deleting a task. This was an important improvement because deleting a task also deletes all schedules linked to it. The confirmation message warns the user about the effect of the action and states that it cannot be undone. This reduces the possibility of accidental data loss and makes the application safer to use.

Empty states were also included. When no tasks exist, the task table displays an empty message instead of showing a blank area. Similarly, when schedules or logs are not available, the interface can communicate that no records are present. Empty states make the application feel complete and informative even when there is no data.

The application also includes loading states for data sections. When schedules or records are being loaded, a loading panel can be displayed. This helps users understand that data is being fetched or processed. Without loading states, users may assume that the application is broken or that no data exists.

The runtime activity chart in the Admin Dashboard improves visual understanding. Instead of showing only text-based information, the chart provides a quick view of

execution activity. Visual elements make it easier for administrators to observe patterns and identify possible issues.

The responsive layout was another important improvement. The interface was designed to remain usable on different screen sizes. Forms, cards, tables, and panels adjust to the available screen space. This makes the application usable on desktops, laptops, and smaller screens. Responsive behavior is useful because users may access the system from different devices during review or operational use.

Overall, the UI/UX improvements made the application more user-friendly and professional. The system provides clear structure, immediate feedback, safer actions, useful visual summaries, and better responsiveness. These improvements support both project demonstration and real-world usability.

6.5 Observations and Outcomes

The implementation successfully demonstrated the main workflow of a scheduled-task self-service utility. The application allowed business users to create tasks, attach batch file details, configure schedules, pause and resume tasks, delete records, and troubleshoot issues through logs. It also allowed admin users to monitor overall system health using metrics, charts, tables, and log views.

One of the main observations was that the project provides a strong frontend foundation. The code structure is modular and organized into reusable components, dashboard pages, layout components, hooks, mock API services, and domain types. This separation makes the system easier to understand, maintain, and extend. Each part of the application has a clear role, which improves long-term maintainability.

Another important outcome was that the mock API successfully supported complete frontend testing. Even without a real backend, the application could simulate task creation, schedule creation, pause and resume actions, deletion behavior, logs, and metrics. This allowed the project to demonstrate realistic workflows during review. The mock API can later be replaced with real C# backend endpoints without changing most of the frontend components.

The system also demonstrated important operational rules. Task deletion correctly removed linked schedules, while schedule deletion did not remove the parent task. Task status could be changed using pause and resume actions. Major actions generated log entries, which helped create an audit trail. These behaviors matched the expected requirements of a scheduler management tool.

The review also confirmed that the role-based structure was working as expected. Business users were directed toward operational task and schedule management, while administrators were directed toward monitoring and oversight. Restricted access behavior prevented non-admin users from viewing the Admin Dashboard. This showed that the frontend was ready to support real authentication and authorization in the future.

Another observation was that the application improved operational visibility. Users could view logs, filter them by severity, search by task name or message, and identify issues from different sources such as task-manager, scheduler, and batch-runner. This makes the system useful not only for configuration but also for troubleshooting and review.

The user experience was also improved through validation, toast notifications, confirmation dialogs, empty states, loading states, and responsive layout. These features made the application clearer and safer to use. They also made the project more suitable for demonstration because reviewers could easily understand the flow and see feedback after each action.

Some advanced modules were identified as future enhancements. Real authentication, first-time password setup, persistent audit storage, cron expression generation, natural language schedule preview, admin user management, task type management, and export/import migration are not fully implemented in the current frontend prototype. However, the current architecture already supports these additions because the project is built with separated components, reusable hooks, typed domain models, and a replaceable mock API layer.

In conclusion, the results confirmed that the project meets the core frontend objectives. It provides a working prototype for role-based task scheduling, operational monitoring, and troubleshooting. The implemented system reduces manual dependency, improves task visibility, supports role-based access, and prepares the application for future backend integration. The project is suitable for demonstration, review, and further development into a complete production-ready scheduled-task self-service utility.

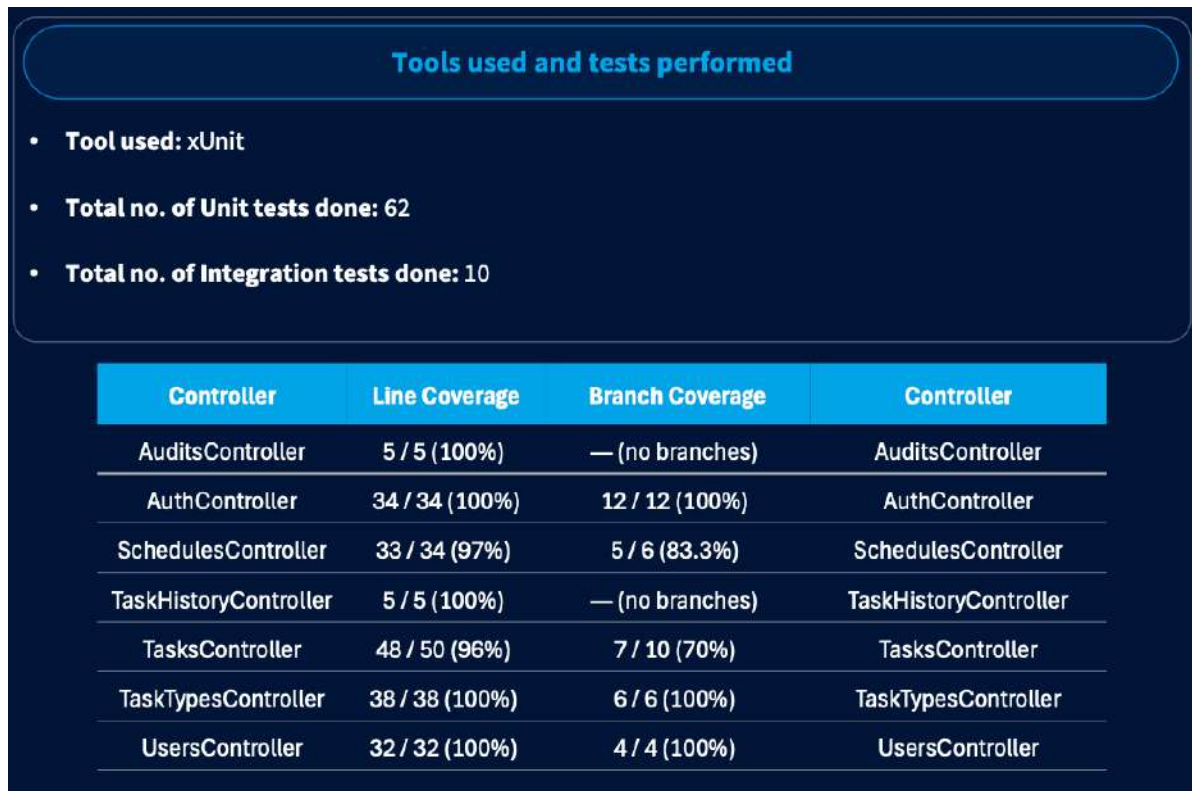


Figure 8: Test Coverage Report

CHAPTER 7

USED TECHNOLOGIES

7.1 React

React was used as the main frontend library for developing the AutoTasker: Self-Service Task Scheduler. It was selected because it allows the application to be built using reusable and independent components. In this project, different parts of the user interface are divided into separate React components such as task forms, schedule forms, task tables, schedule tables, dashboard cards, log viewers, confirmation dialogs, navigation areas, and toast notifications. This component-based structure makes the application easier to develop, test, maintain, and extend.

The project follows a modular frontend structure where each component has a specific responsibility. Page-level components such as `ClientDashboardPage` and `AdminDashboardPage` are responsible for managing complete screens. These pages combine smaller reusable components and arrange them into a complete dashboard layout. Smaller components such as `TaskForm`, `ScheduleForm`, `TaskTable`, `ScheduleTable`, `LogViewer`, and `StatusCards` are responsible for specific user interface sections. This separation improves readability because each file focuses on one part of the system.

React was also useful for managing dynamic user interactions. The application does not behave like a static webpage. It responds to user actions such as creating a task, creating a schedule, pausing a task, resuming a task, deleting a record, searching logs, filtering logs, opening confirmation dialogs, and showing toast notifications. React state management helps handle these interactions smoothly without reloading the entire page.

In this project, React state is used to control form inputs, selected filters, loading indicators, dialog visibility, toast messages, active navigation states, and role-based behavior. For example, when the user submits a task form, the interface can show a loading state while the operation is being processed. After the operation is completed, the task list is updated and a toast notification is displayed. This creates a smooth and interactive user experience.

React also supports conditional rendering, which is important in this project. Conditional rendering means that the interface can show or hide content based on the current state of the application. For example, the Admin Dashboard is shown only when the active role is admin. If the active role is not admin, the application displays a restricted access message. Similarly, the schedule form can be disabled when no task exists, empty messages can be shown when tables do not have data, and loading messages can be shown while records are being fetched.

Another advantage of React is that it makes the frontend scalable. As the project grows, new components can be added without disturbing the existing structure. For example, future modules such as first-time password setup, manage users, manage task types, export/import, and cron builder can be added as new React components and pages. It

allowed the project to be organized as a collection of reusable components and provided the flexibility needed to support role-based dashboards, task operations, schedule operations, logs, and admin monitoring features.

7.2 TypeScript

TypeScript was used in the project to improve code safety, clarity, and maintainability. Since the application handles structured information such as tasks, schedules, logs, dashboard metrics, task statuses, task types, schedule modes, and user roles, it is important to define the expected shape of data clearly. TypeScript helps achieve this by allowing the developer to define types and interfaces for different parts of the application. The project includes domain-level type definitions for important entities. These include `UserRole`, `TaskType`, `TaskStatus`, `ScheduleMode`, `Frequency`, `Task`, `Schedule`, `LogEntry`, and `DashboardMetrics`. These type definitions act as contracts across the project. They ensure that components, hooks, and API services use data in a consistent format.

For example, `UserRole` can be restricted to values such as `business` and `admin`. `TaskType` can be restricted to `T1`, `T2`, `T3`, and `T4`. `TaskStatus` can represent states such as `active`, `paused`, or `error`. Log levels can be restricted to `INFO`, `WARN`, and `ERROR`. These restrictions prevent accidental use of invalid values during development.

TypeScript helps identify mistakes before the application is run in the browser. If a component expects a `Task` object but receives an incomplete object, TypeScript can show an error during development. This reduces runtime errors and improves the reliability of the application. It is especially useful when multiple components depend on the same data structures.

TypeScript also improves code readability. When a developer opens a component or hook, the type definitions make it easier to understand what data the component expects and what it returns. For example, a table component that receives a list of tasks can clearly define that it expects an array of `Task` objects. This makes the code easier to understand for other developers or reviewers.

In this project, TypeScript also supports safer API handling. The current implementation uses a mock API, but the same type definitions can be reused when connecting to a real backend. For example, when the frontend later receives task data from C# API endpoints, the response can be mapped to the existing `Task` type. This helps ensure that the frontend and backend remain consistent.

TypeScript is also useful for maintaining the project in the long term. As new features are added, such as user management, task type configuration, cron expression generation, and export/import, new types can be created and reused throughout the application. This reduces confusion and prevents data mismatch across different modules.

Overall, TypeScript improved the quality of the project by making data structures clear, reducing development errors, improving maintainability, and preparing the frontend for future backend integration.

7.3 Material UI

Material UI is a React component library that provides ready-made user interface components for building professional and responsive applications. It includes components such as buttons, text fields, dropdowns, dialogs, cards, tables, navigation bars, icons, snack bars, and layout utilities. These components follow a consistent design system and can help speed up frontend development.

In the current implementation of the AutoTasker: Self-Service Task Scheduler, custom CSS is used instead of Material UI components. The user interface is styled through `styles.css`. Custom styles are used for panels, forms, buttons, task tables, schedule tables, status cards, dialogs, log sections, layout containers, and responsive behavior. This approach gives complete control over the visual design and keeps the project lightweight. The custom CSS approach was suitable for the current prototype because the project required a clean and focused dashboard design. The interface was designed specifically for scheduled-task management, so custom panels and layouts were created according to the project's needs. This made it possible to maintain a simple design without depending on a large external UI library.

However, Material UI can be integrated in future versions of the project to improve design consistency and development speed. Components such as `TextField`, `Select`, `Button`, `Dialog`, `Snackbar`, `Table`, `Card`, `Tabs`, and `Menu` can be used to replace custom UI elements. This would reduce the amount of custom styling required and provide a more standardized look and feel.

Material UI would be especially useful when adding larger admin modules. For example, `Manage Users` can use Material UI tables, dialogs, forms, chips, and status indicators. `Manage Task Types` can use forms and tables for adding, editing, disabling, and viewing task types. `Export` and `Import` screens can use file upload components, progress indicators, validation result tables, and confirmation dialogs.

Another advantage of Material UI is accessibility. Many Material UI components include built-in accessibility features such as keyboard navigation, proper labels, focus handling, and responsive behavior. This would make the application easier to use for a wider range of users.

Material UI also supports theming. This means the application can define a consistent color palette, typography style, spacing system, and component design. For an enterprise utility like the AutoTasker: Self-Service Task Scheduler, theming can help create a professional and uniform interface across all modules.

Although Material UI is not currently used in the implemented code, it remains a suitable technology option for future enhancement. The current React structure can support Material UI integration because the application is already component-based. Individual custom components can be replaced or upgraded gradually without rewriting the entire project.

7.4 React Router

React Router was used for client-side navigation in the application. It allows users to move between different pages without refreshing the browser. This is important for single-page applications because the frontend should provide a smooth and fast navigation experience.

In this project, React Router is used to define the main routes of the application. The root path redirects to the Client Dashboard. The client route displays the Client Dashboard, and the admin route displays the Admin Dashboard. In text form, the navigation structure is: the root path redirects to /client, /client opens the Client Dashboard, and /admin opens the Admin Dashboard.

The AppShell component contains the common navigation layout. It provides navigation links for moving between the Client Dashboard and Admin Dashboard. It also helps maintain a consistent page structure across the application. Since the navigation is handled on the frontend, switching between dashboard pages is quick and does not require a full browser reload.

React Router also helps highlight the active page. The application can check the current route and visually indicate whether the user is currently on the Client Dashboard or Admin Dashboard. This improves usability because users can easily understand which section of the application they are viewing.

Role-based page control is also supported through the routing structure. Although the admin route exists in the frontend, the AdminDashboardPage checks the active user role before displaying admin content. If the active role is admin, the admin dashboard is shown. If the active role is business, a restricted access message is displayed. This validates the role-based access behavior at the frontend level.

In a production implementation, React Router can be extended with protected route wrappers. A protected route can check whether the user is logged in and whether the user has permission to access a route. If the user is not authenticated, the application can redirect them to the login page. If the user is authenticated but does not have the required role, the application can redirect them to an access denied page or show a restricted access message.

React Router also supports future expansion of the application. Additional routes can be added for first-time password setup, manage users, manage task types, audit history, export/import, cron builder, and task details. Since routing is already established, these pages can be added in an organized way.

Overall, React Router helped create a structured navigation system for the project. It allowed the application to separate the client and admin dashboards, support role-based page display, and prepare the frontend for future protected routes and additional modules.

7.5 Vite

Vite was used as the build tool and development server for the project. It is suitable for modern frontend applications because it provides fast startup, quick refresh, and optimized production builds. Since the project is built using React and TypeScript, Vite provides a simple and efficient development environment.

During development, Vite allows changes made in React components, TypeScript files, and CSS files to appear quickly in the browser. This feature is known as hot module replacement. It improves developer productivity because the developer can immediately see the result of code changes without restarting the application manually.

Vite also makes the project setup simpler. It provides a clean structure for React and TypeScript applications and supports common development commands. The main commands used in the project are `npm run dev`, `npm run build`, and `npm run preview`. The `npm run dev` command starts the local development server. The `npm run build` command creates the production-ready version of the frontend. The `npm run preview` command allows the production build to be tested locally before deployment.

The development server provided by Vite is useful during UI testing. Features such as task creation, schedule creation, role switching, dashboard navigation, log filtering, and responsive layout can be tested quickly in the browser. This helped speed up the implementation and review process.

For production, Vite creates optimized static files. These files can be deployed on static hosting platforms, internal servers, or cloud services. The build output usually contains optimized JavaScript, CSS, and assets that are ready to be served to users.

Vite is also suitable for modular applications. Since this project is divided into components, pages, hooks, services, and domain files, Vite can efficiently handle the build process. It supports modern JavaScript features and works well with TypeScript. Another advantage of Vite is that it keeps the frontend lightweight. The project does not require a heavy build configuration. This makes it easier to run, test, and maintain. For a scheduled-task self-service frontend, this is useful because the focus can remain on functionality and user experience instead of build complexity.

Overall, Vite provided a fast and reliable development environment for the project. It helped in quick testing, smooth development, optimized builds, and future deployment readiness.

7.6 Backend API Integration

The current project uses a mocked API layer to simulate backend behavior. This mock API is located in `mockApi.ts` and provides temporary data and functions for tasks, schedules, logs, and dashboard metrics. Since the backend is not fully connected in the current frontend prototype, the mock API allows the frontend to demonstrate complete

workflows without waiting for backend completion.

The mock API simulates real backend operations such as creating tasks, deleting tasks, creating schedules, deleting schedules, pausing and resuming tasks, fetching logs, and fetching dashboard metrics. It also includes asynchronous delays to imitate real network calls. This makes the frontend behave more realistically during demonstration and testing.

For example, when a user creates a task, the frontend sends the task data to the mock API. The mock API creates a new task object, assigns an ID, sets the status to active, stores the timestamp, and adds the record to the task list. It also generates a log entry for the action. This is similar to how a real backend service would behave.

React Query is used through custom hooks in `useSpectrumData.ts`. These hooks help manage API calls, loading states, data fetching, mutations, and cache updates. The major hooks include `useTasks`, `useSchedules`, `useMetrics`, `useLogs`, and `useTaskActions`. These hooks act as a bridge between the frontend components and the API layer.

This structure keeps the frontend clean. Components do not directly depend on backend implementation details. Instead, components call hooks, and the hooks call the API layer. This makes the application easier to maintain and easier to integrate with a real backend in the future.

When the real backend is implemented, the mock API can be replaced with actual API calls. For example, the frontend can connect to endpoints for getting tasks, creating tasks, deleting tasks, pausing or resuming tasks, getting schedules, creating schedules, deleting schedules, getting logs, and getting dashboard metrics. In text form, these endpoints can include `GET /api/tasks`, `POST /api/tasks`, `DELETE /api/tasks/{taskId}`, `PATCH /api/tasks/{taskId}/pause-toggle`, `GET /api/schedules`, `POST /api/schedules`, `DELETE /api/schedules/{scheduleId}`, `GET /api/logs`, and `GET /api/dashboard/metrics`.

This integration approach is useful because it allows the frontend and backend to be developed separately. The frontend can be completed and reviewed using mock data, while the backend can later provide real database storage, authentication, scheduler execution, audit tracking, and task processing.

The API integration layer can also support error handling. If a backend request fails, the frontend can show an error toast or display a suitable message. Loading states can be shown while API requests are in progress. This improves the overall reliability and user experience of the application.

In the production version, backend integration would also include authentication tokens. After login, the frontend would send the token with API requests. This would make the role-based flow secure beyond the frontend.

Overall, the backend API integration structure makes the project ready for future production development. The use of mock API, React Query hooks, typed domain models, and reusable service functions ensures that real backend endpoints can be connected with minimal changes to the user interface.

7.7 Deployment and Version Control

Version control is managed using Git. Git is used to track changes in the project source code and maintain a history of development. This is important because the project contains multiple files, components, styles, hooks, and API service logic. Git makes it easier to understand what changes were made, when they were made, and why they were made.

Each feature or correction can be committed separately. For example, separate commits can be made for task form development, schedule form development, admin dashboard creation, log viewer implementation, validation improvements, toast notifications, and responsive styling. This makes the development history cleaner and easier to review.

The project can be hosted in a remote repository such as GitHub, GitLab, or Bitbucket. A remote repository provides backup and makes collaboration easier. Team members can clone the project, create branches, make changes, and merge updates. This is useful when multiple developers are working on different modules of the application.

Branching can be used to separate different stages of work. For example, a development branch can be used for active coding, a testing branch can be used for review, and a main branch can be used for stable code. Feature branches can also be created for new modules such as authentication, admin user management, cron builder, and export/import.

For deployment, the Vite production build can be generated using the `npm run build` command. This creates optimized static files that can be deployed to a web server or hosting platform. The build output can be hosted on platforms such as Netlify, Vercel, Azure Static Web Apps, AWS S3, or an internal company server.

Before deployment, the production build can be tested locally using the `npm run preview` command. This helps verify whether the application works properly after the build process. It is important because sometimes an application may work in development mode but face issues after production optimization.

Environment-specific configuration can be managed using environment variables. For example, the development environment can connect to a local API server, the testing environment can connect to a test backend, and the production environment can connect to the live backend. This avoids hardcoding backend URLs inside the application.

Deployment also supports future backend integration. Once real C# APIs are available, the frontend can be configured to call the correct backend URL depending on the environment. This makes the application suitable for development, testing, staging, and production usage.

Overall, Git and deployment tooling help make the project maintainable and release-ready. Git supports source code tracking and collaboration, while Vite build tools support optimized deployment. Together, they make it easier to develop, test, review, deploy, and upgrade the AutoTasker: Self-Service Task Scheduler across different environments.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The AutoTasker: Self-Service Task Scheduler was developed to simplify the creation, scheduling, monitoring, and troubleshooting of server batch tasks using a real-time frontend interface. The main objective of the project was to provide business users with a self-service platform while also giving administrators a separate dashboard for monitoring task activity and system health.

The application successfully implements the core workflow of task and schedule management. Business users can create tasks, assign task types, map batch files, configure schedules, pause or resume tasks, delete schedules, and remove tasks when required. The system also handles important rules such as deleting all related schedules when a task is deleted, while keeping the parent task intact when only a schedule is deleted.

The application uses real-time data communication through backend API integration. Task records, schedule details, execution logs, audit history, and dashboard metrics are fetched from backend services and displayed dynamically in the user interface. This allows users to view the latest task status, schedule information, and troubleshooting output without relying on static or temporary data.

The role-based dashboard structure was also achieved. The Client Dashboard supports business task operations, while the Admin Dashboard provides monitoring cards, runtime activity, task and schedule tables, and troubleshooting logs. This separation improves usability and ensures that each user type sees only the features relevant to their responsibility.

The project uses React, TypeScript, Vite, React Router, backend API services, validation, and reusable UI components. The frontend is modular, maintainable, and suitable for enterprise-level expansion. Overall, the project meets its primary goal of providing a working real-time self-service task scheduling interface.

8.2 Limitations

Although the project implements the main workflow, some limitations still exist. First, the system depends on backend API availability. If the backend service or database is unavailable, real-time task records, schedule data, logs, and metrics cannot be loaded correctly.

Second, authentication and authorization must be handled carefully in production. User login, role validation, password security, session handling, and access control should be protected through secure backend mechanisms.

Third, the application depends on correct scheduler integration. If the scheduler service fails to create, update, pause, resume, or delete jobs, the frontend may show user actions as submitted, but the backend must ensure that the actual scheduled task operation is completed successfully.

Fourth, audit and task history accuracy depends on proper backend logging. Every create, update, delete, pause, resume, and execution event must be recorded consistently to support troubleshooting and accountability.

Fifth, advanced admin modules such as Manage Users, Manage Task Types, and Export/Import may require further enhancement depending on organizational requirements.

Finally, complex scheduling options such as advanced cron expressions, holiday calendars, dependency-based execution, and retry policies may need further implementation for enterprise-level usage.

8.3 Future Work

Future development can focus on improving the system into a more complete enterprise-ready application.

The first improvement is strengthening backend API integration. The APIs should support reliable task creation, schedule updates, execution tracking, log retrieval, audit history, dashboard metrics, and error handling. API responses should be optimized so the frontend can display real-time status updates efficiently.

The second major enhancement is real authentication and authorization. The application should support secure login, logout, role-based claims, encrypted password storage, first-time password setup, password reset, and session expiry handling.

Another important future feature is deeper scheduler integration. The backend should connect with Windows Task Scheduler or an enterprise scheduling system to create, update, pause, resume, and delete actual scheduled jobs. It should also capture execution results, exit codes, runtime duration, and failure reasons from batch files.

The schedule module can be improved with a cron builder and natural language preview. Users should be able to configure complex schedules while seeing readable descriptions such as “Runs every weekday at 9:00 AM UTC.” This would reduce configuration errors and improve user confidence before saving.

The Admin Dashboard can be expanded with Manage Users and Manage Task Types modules. Administrators should be able to create users, assign roles, reset passwords, activate or deactivate accounts, and maintain approved task categories.

Export and import functionality can also be added for environment migration. This would allow task and schedule configurations to be moved between development, testing,

staging, and production environments using validated JSON or CSV files. Finally, the system can be improved with stronger testing and deployment practices. Unit tests, integration tests, API contract tests, and UI tests should be added. CI/CD pipelines can automate build, test, and deployment processes.

In conclusion, the project provides a strong base for a real-time scheduled task management system. With stronger authentication, scheduler orchestration, persistent audit history, admin configuration modules, and deployment automation, it can evolve into a complete enterprise-ready utility.

REFERENCES

- Git Project. (n.d.-a). About version control. Git SCM. Retrieved June 5, 2026, from <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>
- Git Project. (n.d.-b). Pro Git book. Git SCM. Retrieved June 5, 2026, from <https://git-scm.com/book/en/v2>
- GitHub. (n.d.-a). Understanding GitHub Actions. GitHub Docs. Retrieved June 5, 2026, from <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>
- GitHub. (n.d.-b). Deploying with GitHub Actions. GitHub Docs. Retrieved June 5, 2026, from <https://docs.github.com/en/actions/use-cases-and-examples/deploying/deploying-with-github-actions>
- Material UI Team. (n.d.-a). Material UI: React components. MUI Documentation. Retrieved June 5, 2026, from <https://mui.com/material-ui/>
- Material UI Team. (n.d.-b). Getting started. MUI Documentation. Retrieved June 5, 2026, from <https://mui.com/material-ui/getting-started/>
- MDN Contributors. (n.d.-a). Fetch API. MDN Web Docs. Retrieved June 5, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Contributors. (n.d.-b). Using the Fetch API. MDN Web Docs. Retrieved June 5, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- Microsoft. (n.d.-a). Create web APIs with ASP.NET Core. Microsoft Learn. Retrieved June 5, 2026, from <https://learn.microsoft.com/en-us/aspnet/core/web-api/>
- Microsoft. (n.d.-b). Microsoft REST API guidelines. GitHub. Retrieved June 5, 2026, from <https://github.com/microsoft/api-guidelines>

React Router Team. (n.d.-a). Routing. React Router Documentation. Retrieved June 5, 2026, from <https://reactrouter.com/start/declarative/routing>

React Router Team. (n.d.-b). Route. React Router Documentation. Retrieved June 5, 2026, from <https://reactrouter.com/docs/en/en/v6/route/route>

React Team. (n.d.-a). Built-in React hooks. React.dev. Retrieved June 5, 2026, from <https://react.dev/reference/react/hooks>

React Team. (n.d.-b). State: A component's memory. React.dev. Retrieved June 5, 2026, from <https://react.dev/learn/state-a-components-memory>

TanStack. (n.d.-a). Query invalidation. TanStack Query Documentation. Retrieved June 5, 2026, from <https://tanstack.com/query/latest/docs/framework/react/guides/query-invalidation>

TanStack. (n.d.-b). React Query documentation. TanStack Query. Retrieved June 5, 2026, from <https://tanstack.dev/query/v5/docs/framework/react>

TypeScript Team. (n.d.-a). The TypeScript handbook. TypeScript Documentation. Retrieved June 5, 2026, from <https://www.typescriptlang.org/docs/handbook/intro>

TypeScript Team. (n.d.-b). TypeScript for the new programmer. TypeScript Documentation. Retrieved June 5, 2026, from <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html>

Vite Team. (n.d.-a). Getting started. Vite Documentation. Retrieved June 5, 2026, from <https://vite.dev/guide/>

Vite Team. (n.d.-b). Building for production. Vite Documentation. Retrieved June 5, 2026, from <https://vite.dev/guide/build>

Zod. (n.d.-a). Zod documentation. Zod. Retrieved June 5, 2026, from <https://zod.dev/>

Zod. (n.d.-b). Defining schemas. Zod. Retrieved June 5, 2026, from <https://zod.dev/api>